

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	7
1 ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ МОДЕЛИРОВАНИЯ СОСТОЯНИЯ ТЕХНОЛОГИЧЕСКОГО ОБОРУДОВАНИЯ	10
1.1 Обзор исследований и моделей машинного обучения для решения задач предметной области	10
1.2 Критерии оценки и валидации генеративных моделей в промышленной диагностике	17
1.3 Выводы	21
2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ МОДЕЛИРОВАНИЯ ДАННЫХ .	23
2.1 Постановка задачи	23
2.2 Математическая модель	29
2.3 Выводы	33
3 ПРОВЕДЕНИЕ ЭКСПЕРИМЕНТОВ И АНАЛИЗ РЕЗУЛЬТАТОВ .	35
3.1 Постановка эксперимента с архитектурой LSTM-VAE	35
3.2 Постановка эксперимента с архитектурой transformer-VAE	37
3.3 Постановка эксперимента с архитектурой VRNN-VAE	39
3.4 Постановка эксперимента с архитектурой SSM-VAE	40
3.5 Постановка эксперимента с архитектурой МАМБА-VAE	42
3.6 Выводы	44
4 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ МОДЕЛИРОВАНИЯ	46
4.1 Архитектура программной системы	46
4.2 Структура python-библиотеки и организация пакетов	47
4.3 Модуль чтения и обработки данных	49
4.4 Модуль предобработки и конвейера данных	52
4.5 Модуль нейросетевых моделей	57
4.6 Модуль оценки метрик	64
4.7 Модуль интеграции с сервисом MLFlow и управления экспериментами	67
4.8 Описание применения модулей конвейера проведения экспериментов	71
4.9 Установка и развертывание системы	72
4.10 Аппаратные и программные требования к работоспособности системы	74
4.11 Направления и возможности функционирования системы	75
4.12 Выводы	75

ЗАКЛЮЧЕНИЕ	78
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	80

ВВЕДЕНИЕ

На современных этапах развития технологического производства существует потребность в моделировании данных сложного технологического оборудования в условиях наличия зашумлений потоковых данных. Крайне важно не только учитывать текущее состояние составляющих систему модулей, но и уметь прогнозировать их поведение на несколько эксплуатационных циклов вперед. Прогнозирование выхода параметров за допустимые границы позволяет не только снизить риски аварийной эксплуатации, но и повысить общую эффективность производственного процесса, предотвращая преждевременное снятие с эксплуатации ещё ресурсных узлов.

Альтернативным подходом является предиктивное обслуживание, основанное на непрерывном мониторинге фактического технического состояния оборудования и прогнозировании момента наступления отказа. Точное моделирование режимов работы оборудования позволяет оптимизировать графики технического обслуживания, минимизировать незапланированные простои и снизить совокупную стоимость владения сложными техническими системами.

Актуальность данной работы обусловлена необходимостью разработки и экспериментального обоснования системы моделирования рабочих циклов сложного технологического оборудования, в основе которой лежат нейросетевые генеративные модели, в условиях наличия зашумленности в данных.

В связи с этим целью данной работы является снижение эксплуатационных рисков за счет создания системы моделирования состояния оборудования.

Указанная цель реализуется посредством выполнения указанных задач:

- изучение существующих решений поставленной задачи;
- обзор, сравнение и выбор семейства архитектур нейросетевых генеративных моделей;
- построение математической модели предсказания будущих циклов работы оборудования;
- проведение серии экспериментов на оценку устойчивости генерации

показателей датчиков при наличии шумов и пропусков в исходных данных;

- построение программной системы на основе выбранных моделей.

Научная новизна работы заключается в создании генеративной нейросетевой модели, предназначенной для моделирования значений датчиков технологического оборудования. В отличие от аналогичных, предлагаемая модель предназначена для работы в условиях наличия шумов и пропусков во входных данных и показывает удовлетворительное качество генерации в широких пределах изменений зашумления показателей датчиков.

Математический аппарат алгоритма базируется на адаптации вариационной нижней оценки (ELBO) под задачу последовательного прогнозирования. Вероятностная природа модели обеспечивает встроенный механизм фильтрации высокочастотных шумов: латентное пространство обучается представлять низкочастотные тренды деградации, отбрасывая случайные отклонения, обусловленные погрешностями измерений.

Теоретическая значимость работы заключается в предложенной математической модели генеративной архитектуры на базе вариационного автокодировщика и формализации способа учета шумов.

Практическая значимость данной работы заключается в построении программной системы в виде python-библиотеки, которая может быть использована для моделирования значений детекторов технологического оборудования. Данная система включает в себя следующие элементы:

- пакет загрузки данных и их обработки;
- пакет использования и создания нейросетевых моделей;
- пакет расчета метрик и статистического анализа моделей;
- модуль с готовыми конвейерами обработки входной информации;
- пакет для загрузки и сохранения экспериментов в сервисе MLFlow;
- пакет визуализаций экспериментов (обучение модели, инференс).

Положение выносимое на защиту: генеративная модель на базе вариационного автокодировщика с кодировщиком VRNN способна генерировать показания датчиков при наличии зашумлений с падением точности не более двадцати процентов.

Публикации автора по теме диссертационного исследования.

1) Захаров, Д.А. Нейросетевой программный модуль оценки состояния технологического оборудования / Д.А. Захаров // XXIX Региональная

конференция молодых ученых и исследователей Волгоградской области (г. Волгоград, 16 сентября – 15 ноября 2024 г.) : сб. материалов конф. / редкол.: С. В. Кузьмин (отв. ред.) [и др.]; ВолгГТУ [и др.]. - Волгоград, 2024. - С. 131-132.

В первой главе проводится анализ существующих методов решения поставленной задачи. Рассматриваются базовые подходы к построению архитектуры на базе вариационного автокодировщика.

Во второй главе описывается проектирование системы и постановка задачи. Приводятся математическое описание исследуемых архитектур моделей.

В третьей главе описывается проведение экспериментов. Проводится анализ результатов проделанного исследования.

В четвертой главе приводится описание построения системы моделирования данных.

1 ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ МОДЕЛИРОВАНИЯ СОСТОЯНИЯ ТЕХНОЛОГИЧЕСКОГО ОБОРУДОВАНИЯ

Оценка и моделирование текущего состояния оборудования являются ключевыми факторами перехода к предиктивному обслуживанию. Своевременное формирование вероятностных сценариев развития состояния дает возможность использовать оборудование с максимальной эффективностью и оптимизировать графики технического обслуживания. В работах, рассматриваемых в первой главе, проводится анализ решений задач прогнозирования остаточного ресурса технологического оборудования с использованием моделей глубокого обучения. Особое внимание в современных исследованиях уделяется генеративным архитектурам, способным моделировать распределения временных рядов и синтезировать правдоподобные траектории развития параметров. При этом одной из ключевых проблем, ограничивающих применение существующих подходов в реальных промышленных условиях, является их чувствительность к шумам измерений, погрешностям датчиков и нестационарности эксплуатационных режимов.

1.1 Обзор исследований и моделей машинного обучения для решения задач предметной области

1.1.1 Применение классических и глубоких автокодировщиков для оценки состояния оборудования

В работе [1] под названием “An Anomaly Detection Framework Based on Autoencoder and Nearest Neighbor” исследователи предлагают архитектуру АЕКNN, объединяющую автокодировщик и метод k -ближайших соседей. На этапе обучения модель тренируется исключительно на данных штатного режима. Сжатое представление в латентном слое используется как вход для k -NN классификатора. При тестировании аномалии выявляются по отклонению расстояния до ближайших соседей в скрытом пространстве. Эффективность подхода подтверждается на промышленных датасетах, однако метод не предусматривает генерацию будущих состояний и работает в режиме классификации «штатный/аварийный» без оценки вероятностной

неопределенности.

В статье [2] “Обнаружение аномальных событий на хосте с использованием автокодировщика” авторы предлагают метод построения модели нормального поведения системы на основе двух параллельных автокодировщиков с различной архитектурой. Критерием аномальности выступает превышение порога мгновенной ошибки реконструкции, рассчитываемого отдельно для каждого декодировщика. Метод демонстрирует высокую чувствительность к редким событиям, но опирается на детерминированную реконструкцию, что ограничивает его применимость для стохастических процессов износа технологического оборудования.

1.1.2 Вариационные автокодировщики в задачах вероятностного прогнозирования и генерации временных рядов

Применение вариационных автокодировщиков (VAE) для моделирования временных рядов представляет собой активно развивающееся направление исследований на стыке генеративного моделирования и анализа последовательностей. В отличие от классических детерминированных архитектур, VAE позволяют оценивать неопределённость прогноза и генерировать правдоподобные траектории развития параметров, что критически важно для задач предиктивного анализа. В данном разделе рассмотрены фундаментальные работы, формирующие методологическую основу предлагаемого подхода.

1.1.2.1 Фундаментальная модель вариационного автокодировщика

Работа [2] под названием “Auto-Encoding Variational Bayes” является основополагающей в области генеративного моделирования и вводит формальный математический аппарат вариационного автокодировщика. Авторы предлагают подход к обучению глубоких генеративных моделей путём максимизации вариационной нижней оценки (Evidence Lower Bound, ELBO) логарифма правдоподобия наблюдаемых данных.

Математическая суть метода заключается в следующем. Пусть X — наблюдаемые данные, а z — латентные переменные. Прямое вычисление маргинального правдоподобия $p_{\theta}(X) = \int p_{\theta}(X | z)p(z)dz$ является аналитически неразрешимой задачей для сложных моделей. Вместо этого

вводится аппроксимирующее распределение $q_\phi(z \mid X)$, параметризуемое нейронной сетью (кодировщиком), и максимизируется нижняя оценка:

$$\mathcal{L}(\theta, \phi; X) = \mathbb{E}_{q_\phi(z|X)} [\log p_\theta(X \mid z)] - D_{KL}(q_\phi(z \mid X) \parallel p(z)). \quad (1)$$

Первый член отвечает за точность реконструкции, второй — за регуляризацию латентного пространства относительно априорного распределения $p(z)$, обычно выбираемого как стандартное нормальное $\mathcal{N}(0, I)$. Ключевым техническим вкладом работы является метод репараметризации, позволяющий дифференцировать операцию сэмплирования из распределения и тем самым применять стохастический градиентный спуск для оптимизации параметров ϕ кодировщика.

Авторы подчеркивают преимущества данного подхода: непрерывность и гладкость латентного пространства, обеспечивающая возможность интерполяции между состояниями и генерации новых правдоподобных выборок. Регуляризация через дивергенцию Кульбака-Лейблера предотвращает переобучение и гарантирует, что латентные представления остаются семантически осмысленными.

Авторами была установлено, что классическая архитектура VAE имеет ряд ограничений применительно к задачам временных рядов. Во-первых, модель предполагает независимость наблюдений, что противоречит природе последовательных данных, где каждое наблюдение обусловлено предыдущими. Во-вторых, стандартный VAE оперирует фиксированной размерностью входа и не способен генерировать последовательности переменной длины. В-третьих, априорное распределение $p(z)$ выбирается независимо от входных данных, что ограничивает выразительную мощность модели при работе со сложными многомерными временными рядами телеметрии.

Несмотря на указанные ограничения, работа [2] заложила теоретический фундамент, на котором строятся все последующие модификации VAE для временных рядов.

1.1.2.2 Рекуррентный вариационный автокодировщик для многомерных временных рядов

В работе [3], с наименованием “A Recurrent Latent Variable Model for Sequential Data”. предложена архитектура VRNN (Variational Recurrent Neural Network), объединяющая преимущества рекуррентных нейронных сетей и вариационных автокодировщиков для моделирования многомерных временных рядов с пропусками в данных. Авторы решают ключевую проблему классического VAE — отсутствие явного учёта временных зависимостей — путём введения рекуррентного скрытого состояния h_t , которое передаётся как в кодировщик, так и в декодировщик на каждом шаге времени.

Формально модель определяется следующими уравнениями:

$$q_\phi(z_t | x_t, h_{t-1}) = \mathcal{N}(\mu_\phi(x_t, h_{t-1}), \sigma_\phi^2(x_t, h_{t-1})), \quad (2)$$

$$p_\theta(x_t | z_t, h_{t-1}) = \mathcal{N}(\mu_\theta(z_t, h_{t-1}), \sigma_\theta^2(z_t, h_{t-1})), \quad (3)$$

$$h_t = f_{\text{RNN}}(h_{t-1}, x_t, z_t), \quad (4)$$

где f_{RNN} — функция перехода рекуррентной сети (например, LSTM или GRU). Функция потерь принимает вид суммы по всем временным шагам:

$$\mathcal{L} = \sum_{t=1}^T (\mathbb{E}_{q_\phi(z_t|x_t, h_{t-1})} [\log p_\theta(x_t | z_t, h_{t-1})] - D_{KL}(q_\phi(z_t | x_t, h_{t-1}) \| p_\theta(z_t | h_{t-1}))). \quad (5)$$

Авторы подчеркивают, что главным достоинством VRNN является способность моделировать сложные нелинейные зависимости во времени и генерировать правдоподобные многошаговые траектории. Амортизированный вывод параметров латентного распределения z_t по скользящему окну данных позволяет эффективно агрегировать контекст временного ряда и фильтровать высокочастотный шум измерений. Эксперименты на синтетических и реальных данных (включая речевые сигналы и рукописные символы) демонстрируют превосходство VRNN над классическими RNN и стандартными VAE в задачах генерации и дополнения

временных последовательностей.

Также, авторами установлено, что архитектура VRNN имеет существенные недостатки для задач предиктивного обслуживания. Во-первых, векторное представление состояния z_t на каждом временном шаге увеличивает вычислительную нагрузку и усложняет интерпретацию результатов — оператору системы сложно понять физический смысл многомерного латентного вектора. Во-вторых, рекуррентная структура приводит к накоплению ошибок при многошаговом прогнозировании, что критично при оценке остаточного ресурса на длительных горизонтах. В-третьих, модель требует значительных объёмов размеченных данных для обучения, что противоречит условию дефицита информации об аварийных режимах в реальных промышленных системах.

Несмотря на указанные ограничения, идея объединения рекуррентной памяти и вариационного вывода является концептуально важной для предлагаемого в диссертации подхода, где амортизированный вывод параметров латентного распределения по скользящему окну из n циклов позволяет сохранить преимущества VRNN, избегая при этом проблемы накопления ошибок.

1.1.2.3 Применение вариационного автокодировщика для прогнозирования остаточного ресурса

Работа [4] под названием “Variational Autoencoder for Remaining Useful Life Prediction” представляет собой наиболее близкое к теме диссертации исследование, в котором вариационный автокодировщик применяется непосредственно для задачи прогнозирования остаточного ресурса технологического оборудования. Авторы предлагают архитектуру, в которой VAE обучается на нормальных режимах работы двигателя, а отклонение латентных представлений тестовых данных от распределения нормы используется как индикатор деградации.

Методология исследования включает несколько ключевых этапов. На этапе обучения модель оптимизируется исключительно на данных штатной эксплуатации, что соответствует постановке задачи обнаружения аномалий. Латентное пространство при этом структурируется так, что нормальные состояния концентрируются в центральной области, а аномальные

отклонения формируют периферийные кластеры. Для прогнозирования остаточного ресурса авторы вводят регрессионный слой поверх латентных представлений, обучаемый на доступных размеченных данных об отказах.

Экспериментальная проверка выполнена на датасете NASA C-MAPSS (подмножество FD001), что обеспечивает корректное сравнение с альтернативными методами. Результаты демонстрируют, что предложенный подход достигает сопоставимой с методами контролируемого обучения точности прогнозирования остаточного ресурса, при существенно меньших требованиях к объёму размеченных данных. В частности, значение функции потерь NASA Score для VAE-подхода составило 312 против 298 у базовой архитектуры LSTM, обученной на полном размеченном датасете.

Ключевым преимуществом работы является демонстрация принципиальной возможности решения задачи прогнозирования отказов в условиях дефицита размеченных данных. Вероятностная природа модели позволяет оценивать неопределённость прогноза через дисперсию латентного распределения, что критически важно для принятия решений в условиях риска.

Однако предложенный подход имеет ряд методологических ограничений. Во-первых, использование регрессионного слоя поверх латентных представлений требует хотя бы частичной разметки данных об отказах, что не всегда выполнимо на практике. Во-вторых, модель не учитывает многошаговую природу прогнозирования — прогноз остаточного ресурса формируется для каждого цикла независимо, без учёта временной согласованности траектории деградации. В-третьих, отсутствует явный механизм калибровки порогов обнаружения аномалий через эмпирическую функцию распределения, что снижает интерпретируемость результатов для операторов систем мониторинга.

Анализ рассматриваемой работы подтверждает актуальность применения вариационных автокодировщиков для задач предиктивного обслуживания и выявляет направления для дальнейшего развития метода, включая многошаговое прогнозирование, обучение исключительно на данных нормы и формирование интерпретируемых скалярных индикаторов состояния — именно те аспекты, которые развиваются в предлагаемом в диссертации подходе.

1.1.2.4 Сравнительный анализ рассмотренных подходов

Сопоставление трёх рассмотренных работ позволяет выделить эволюцию применения архитектуры вариационного автокодировщика в задачах временных рядов. Фундаментальная модель, предложенная в работе [1] обеспечивает теоретическую основу, но не учитывает временную структуру данных. Архитектура VRNN-Chung решает эту проблему путём введения рекуррентного скрытого состояния, однако ценой увеличения вычислительной сложности и снижения интерпретируемости. Рассматриваемая работа адаптирует вариационный автокодировщик непосредственно для задачи прогнозирования остаточного ресурса, но сохраняет зависимость от размеченных данных и не использует многошаговое прогнозирование.

Предлагаемый в диссертации подход синтезирует преимущества рассмотренных методов: вероятностная регуляризация латентного пространства [1], амортизированный вывод по скользящему окну [2] и ориентация на задачу предиктивного обслуживания [3], при этом устраняя их ключевые недостатки — зависимость от разметки, проблему накопления ошибок и низкую интерпретируемость результатов.

В работе [5] заложены теоретические основы архитектуры вариационного автокодировщика, где максимизация вариационной нижней оценки (ELBO) обеспечивает одновременное обучение кодировщика, декодировщика и регуляризацию латентного пространства. Применительно к телеметрии оборудования данная архитектура позволяет оценивать не только точечное восстановление параметров, но и дисперсию реконструкции, что напрямую связано с надежностью прогноза.

Авторы исследования [6] демонстрируют применение рекуррентного VAE (VRNN) для многошагового прогнозирования. Авторы показывают, что амортизированный вывод параметров распределения z по скользящему окну данных позволяет генерировать правдоподобные траектории развития параметров. Однако в работе используется векторное представление состояния на каждом шаге, что увеличивает вычислительную нагрузку и усложняет интерпретацию скалярных индикаторов работоспособности.

1.1.3 Сравнительный анализ генеративных архитектур для моделирования рабочих процессов

В обзоре Goodfellow [7] по генеративным состязательным сетям и последующих работах по диффузионным моделям отмечается высокое качество генерации сложных распределений. Однако для задач прогнозирования циклов технологического оборудования эти архитектуры демонстрируют существенные ограничения. Так например архитектуры на основе GAN страдают от нестабильности обучения и отсутствия явной вероятностной интерпретации латентных переменных, что затрудняет калибровку порогов аварийных состояний. Диффузионные модели требуют многоэтапного обратного процесса генерации, что критично увеличивает время инференса и делает их неоптимальными для систем мониторинга в реальном времени. В работе [8] под названием “Learning Structured Output Representation using Deep Conditional Generative Models” показано, что условные VAE (CVAE) обеспечивают стабильную оптимизацию и явную связь между входным контекстом (текущие n циклов) и выходной последовательностью, что позволяет формировать прогнозы с оценкой доверительных интервалов, что напрямую соответствует требованиям к системам предиктивного обслуживания.

1.2 Критерии оценки и валидации генеративных моделей в промышленной диагностике

Оценка качества генеративных моделей представляет собой самостоятельную методологическую задачу, отличающуюся от валидации классических алгоритмов контролируемого обучения. Если для регрессионных и классификационных моделей достаточно стандартных метрик (MSE, Ассигасу, F1), то для генеративных архитектур необходимо одновременно оценивать несколько аспектов: точность реконструкции, качество латентного пространства, правдоподобие генерируемых выборок и устойчивость к зашумлённым данным. В задачах промышленной диагностики эти требования дополняются необходимостью статистической валидации результатов и интерпретируемости метрик для операторов систем

мониторинга. В данном разделе рассмотрены три работы, формирующие методологическую основу оценки предлагаемого подхода.

1.2.1 Обзор метрик оценки генеративных моделей

Работа [9] под названием “Pros and Cons of GAN Evaluation Measures” представляет собой наиболее полный на сегодняшний день обзор метрик оценки генеративных моделей, включая вариационные автокодировщики и генеративно-состязательные сети.

Ключевым результатом работы является классификация метрик на три группы. Первая группа — метрики точности реконструкции, включающие среднеквадратическую ошибку (MSE) и структурное сходство (SSIM). MSE определяется как:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2, \quad (6)$$

где x_i — исходные данные, $\hat{x}_i$ — реконструированные моделью данные, N — размерность выборки.

Вторая группа — метрики качества латентного пространства, среди которых особое место занимает вариационная нижняя оценка ELBO:

$$\mathcal{L} = \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x | z)] - D_{KL}(q_\phi(z | x) \| p(z)). \quad (7)$$

Третья группа — метрики качества генерации, включая Frechet Inception Distance (FID) и Inception Score (IS). FID вычисляется как расстояние Фреше между распределениями признаков реальных и сгенерированных данных:

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{Tr} \left(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2} \right), \quad (8)$$

где μ_r, Σ_r — среднее и ковариация признаков реальных данных, μ_g, Σ_g — аналогичные характеристики для сгенерированных выборок.

Автор показывает, что ни одна из существующих метрик не является универсальной. MSE чувствительна к выбросам, но не учитывает структурные особенности данных. ELBO обеспечивает нижнюю границу правдоподобия, но её абсолютные значения сложно интерпретировать. В

работе демонстрируется высокое качество генерации, для которого требуется большие вычислительные ресурсы и предобученная сеть классификатора.

Автор подчеркивает, что для задач промышленной диагностики наиболее релевантными являются метрики первой и второй групп, поскольку они непосредственно связаны с точностью восстановления телеметрических сигналов и структурированностью латентного пространства, используемого для детектирования аномалий.

1.2.2 Валидация моделей прогнозирования остаточного ресурса

Работа [10] под названием “Remaining useful life estimation in prognostics using deep convolution neural networks” посвящена методологии валидации моделей прогнозирования остаточного ресурса функции потерь, учитывающие асимметрию рисков в задачах предиктивного обслуживания.

Основной вклад работы заключается в систематизации протоколов валидации. Авторы выделяют три уровня оценки. Первый уровень — метрики точности прогноза:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |\text{RUL}_{\text{pred},i} - \text{RUL}_{\text{true},i}|, \quad (9)$$

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\text{RUL}_{\text{pred},i} - \text{RUL}_{\text{true},i})^2}. \quad (10)$$

Второй уровень — специализированная метрика NASA Score, учитывающая асимметрию штрафов за ранний и поздний прогноз:

$$\text{Score} = \sum_{i=1}^N \begin{cases} e^{-d_i/13} - 1, & \text{если } d_i < 0 \\ e^{d_i/10} - 1, & \text{если } d_i \geq 0, \end{cases} \quad (11)$$

где $d_i = \text{RUL}_{\text{pred},i} - \text{RUL}_{\text{true},i}$ — ошибка прогноза для i -го двигателя.

Третий уровень — статистическая валидация через коэффициент детерминации R^2 :

$$R^2 = 1 - \frac{\sum_{i=1}^N (\text{RUL}_{\text{true},i} - \text{RUL}_{\text{pred},i})^2}{\sum_{i=1}^N (\text{RUL}_{\text{true},i} - \overline{\text{RUL}}_{\text{true}})^2}. \quad (12)$$

Авторы указывают, что для корректного сравнения моделей необходимо использовать все три уровня одновременно. Оптимизация только по MSE может привести к переобучению на средних значениях и потере способности предсказывать критические режимы. Использование NASA Score в качестве единственной целевой функции, напротив, может привести к чрезмерно консервативным прогнозам.

Экспериментальная проверка выполнена на подмножествах FD001 и FD003 датасета C-MAPSS. Результаты показывают, что комплексный протокол валидации позволяет выявить слабые места моделей, которые не обнаруживаются при использовании отдельных метрик. В частности, модель с лучшим значением MSE может демонстрировать худший NASA Score, что свидетельствует о её неспособности адекватно оценивать риски поздних прогнозов.

Для задач предиктивного обслуживания, рассматриваемых в диссертации, указанный подход является методологической основой, поскольку обеспечивает статистически обоснованное сравнение предлагаемого VAE-подхода с альтернативными архитектурами.

1.2.3 Валидация моделей детектирования аномалий с использованием временных рядов

Работа [11] посвящена методологии оценки моделей детектирования аномалий в многомерных временных рядах с применением рекуррентных нейронных сетей. Авторы исследуют проблему калибровки пороговых значений и предлагают протокол валидации, учитывающий специфику несбалансированных данных, характерную для задач промышленной диагностики.

Ключевым результатом работы является демонстрация недостаточности метрики Ассигасу для оценки моделей детектирования аномалий. В условиях, когда доля аномальных точек составляет менее 1% от общего объёма данных, тривиальный классификатор, всегда предсказывающий норму, демонстрирует Ассигасу свыше 99%, но при этом полностью бесполезен на практике. Авторы предлагают использовать комбинацию метрик:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad (13)$$

$$\text{Recall} = \frac{TP}{TP + FN}, \quad (14)$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \quad (15)$$

где TP — истинно положительные срабатывания, FP — ложные срабатывания, FN — пропущенные аномалии.

Для оценки качества модели при различных пороговых значениях используется площадь под ROC-кривой (AUC-ROC):

$$\text{AUC-ROC} = \int_0^1 \text{TPR}(\text{FPR}^{-1}(x)) dx, \quad (16)$$

где $\text{TPR} = \text{Recall}$ — доля истинно положительных срабатываний, $\text{FPR} = FP/(FP + TN)$ — доля ложных срабатываний.

Авторы показывают, что для многомерных временных рядов телеметрии критически важна не только точность детектирования, но и временная согласованность срабатываний. Модель, генерирующая серию ложных срабатываний подряд, снижает доверие операторов и может привести к игнорированию реальных аномалий.

1.3 Выводы

В рамках первой главы выполнен анализ предметной области прогнозирования технического состояния сложного оборудования и проведен сравнительный обзор существующих методов машинного обучения. На основе проведенного исследования сформулированы следующие основные результаты.

Установлено, что традиционные детерминированные методы прогнозирования не обеспечивают оценки неопределенности прогноза, что критично для задач мониторинга ответственных узлов. Генеративные состязательные сети и диффузионные модели, несмотря на высокую точность генерации, обладают значительной вычислительной сложностью, затрудняющей их применение в системах реального времени.

В результате анализа было установлено, что на данный момент не проверена концепция многошагового прогнозирования на предмет

устойчивости к шумам во входных данных.

Полученные теоретические и методологические результаты создают необходимую базу для перехода к практической реализации.

2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ МОДЕЛИРОВАНИЯ ДАННЫХ

2.1 Постановка задачи

Требуется спроектировать и реализовать программный комплекс для вероятностного моделирования и прогнозирования состояния оборудования на основе нейросетевых генеративных моделей. Ниже изложены основные требования к системе.

Система должна осуществлять многошаговое прогнозирование телеметрических параметров на n эксплуатационных циклов вперёд с оценкой вероятностной неопределённости.

Реализация системы должна базироваться на архитектуре вариационного автокодировщика для обеспечения моделирования рабочих режимов оборудования.

Система должна принимать на вход временные ряды показаний датчиков в форматах CSV и JSON. Единицей обработки должно выступать скользящее окно из n последовательных циклов.

Модель должна поддерживать модульную интеграцию: загрузку предобученных весов из удалённого хранилища и экспорт предсказаний в форматы CSV и JSON.

Все программные компоненты должны быть оформлены в виде независимых python-пакетов с чётким разделением ответственности (предобработка, обучение, инференс, валидация, сохранение моделей в MLFlow).

В системе должна быть реализована автоматизированная регистрация метрик обучения, и гиперпараметров для обеспечения воспроизводимости экспериментов.

Оценка качества модели должна производиться по комплексу метрик: поточечная MSE, DTW для временных рядов.

Для создания системы предиктивного моделирования состояния технологического оборудования в качестве базового источника данных принят датасет NASA C-MAPSS (Commercial Modular Aero-Propulsion System Simulation), содержащий телеметрические показания турбовентиляторных двигателей в различных режимах эксплуатации и условиях окружающей среды.

2.1.1 Описание входных данных

Исходным набором данных для проведения исследований является двумерный массив полетной телеметрии, содержащий последовательные записи о состоянии набора авиационных двигателей NASA Turbofans (C-MAPSS). Вектор сырых данных для каждого отдельного двигателя представляет собой временную последовательность полетных циклов.

Пусть $\mathbf{X}_{\text{raw}} \in \mathbb{R}^{N_{\text{total}} \times (2+D)}$ — исходная плоская матрица данных, где N_{total} — общее число зарегистрированных полетных циклов (строк) в выборке, размерность столбцов включает в себя два служебных идентификатора, D — количество информационных каналов (операционные настройки и показания физических датчиков). Спецификация столбцов исходной матрицы имеет вид:

$$\mathbf{X}_{\text{raw}} = [\mathbf{u}, \mathbf{c}, \mathbf{f}_1, \mathbf{f}_2, \dots, \mathbf{f}_D], \quad (17)$$

где $\mathbf{u} \in \mathbb{N}^{N_{\text{total}}}$ — вектор идентификаторов контролируемых двигателей (*unit number*), $\mathbf{c} \in \mathbb{N}^{N_{\text{total}}}$ — вектор порядковых номеров полетных циклов (*time in cycles*), а $\mathbf{f}_d \in \mathbb{R}^{N_{\text{total}}}$ — вектор физических измерений d -го информационного канала.

В рамках разработанного конвейера предобработки служебные столбцы ($\mathbf{u}, \mathbf{c}$) изолируются от процедуры внесения искажений, а матрица датчиков подвергается Z-нормализации и процедуре скользящего сглаживания с размером окна в n циклов. Это позволяет сформировать двумерную матрицу стандартизированных значений:

$$\mathbf{X}_{\text{clean}} = \text{StandardScaler}(\text{RollingMean}([\mathbf{f}_1, \dots, \mathbf{f}_D])) \in \mathbb{R}^{N_{\text{total}} \times D}. \quad (18)$$

Для проведения тестирования моделей на устойчивость к отказам оборудования формируется зашумленная матрица входных признаков $\mathbf{X}_{\text{stressed}} = \text{apply_stress_to_data}(\mathbf{X}_{\text{clean}})$. На основе переданного параметра *noise_type* на отмасштабированные датчики накладываются два типа зашумлений:

К каждому элементу исходной матрицы добавляется случайная составляющая, подчиняющаяся нормальному (Гауссову) закону

распределения с нулевым математическим ожиданием и заданным уровнем стандартного отклонения σ_{noise} :

$$x_{\text{stressed},i,d} = x_{\text{clean},i,d} + \epsilon_{i,d}, \quad \epsilon_{i,d} \sim \mathcal{N}(0, \sigma_{\text{noise}}^2). \quad (19)$$

Моделируется случайная потеря пакетов данных с интенсивностью `drop_rate`. Вычисляется доля случайных ячеек матрицы, датчики заменяются на индикатор полного отсутствия сигнала `fill_value`:

$$x_{\text{stressed},i,d} = \begin{cases} 0.0, & \text{с вероятностью } \text{drop_rate}, \\ x_{\text{clean},i,d}, & \text{с вероятностью } 1 - \text{drop_rate}. \end{cases} \quad (20)$$

В комбинированном режиме оба типа зашумления накладываются последовательно, после чего значения принудительно ограничиваются для предотвращения математических выбросов.

На финальном этапе матрицы представляются в виде комбинаций скользящих трехмерных окон с заданной шириной и значением сдвига. В результате формируются две ключевые структуры данных, поступающие на вход нейросети.

Зашумленная предыстория представляется трехмерным тензором $\mathcal{X}_{\text{stressed}} \in \mathbb{R}^{B \times M \times D}$, содержащим зашумленные и отсутствующие сигналы первых n циклов скользящего окна. На основе этой матрицы кодировщик учится строить латентный вектор.

Не зашумленная предыстория представляет собой изолированный вектор $\mathbf{x}_{\text{anchor}} \in \mathbb{R}^{B \times D}$, отмасштабированных значений из сглаженной матрицы $\mathbf{X}_{\text{clean}}$. Данный вектор содержит данные, служащие предысторией, на основании которой декодировщик может генерировать будущие циклы работы. Здесь B — результирующее количество сформированных трехмерных окон после фильтрации граничных циклов двигателей.

2.1.2 Описание выходных данных

Целевым результатом работы системы является генерация вероятностного прогноза будущего состояния оборудования. Выход модели представляет собой последовательность сгенерированных векторов наблюдений для k будущих циклов:

Пусть D — размерность пространства измеряемых признаков, M — длина входного зашумленного окна, а T — полный временной интервал, включающий фазу восстановления и фазу прогнозирования.

Каждый s -й сгенерированный моделью сценарий представляет собой многомерный временной ряд, описываемый матрицей $\hat{\mathbf{Y}}^{(s)}$ размерности $(T \times D)$:

$$\hat{\mathbf{Y}}^{(s)} = \begin{bmatrix} \hat{y}_{1,1}^{(s)} & \hat{y}_{1,2}^{(s)} & \cdots & \hat{y}_{1,D}^{(s)} \\ \hat{y}_{2,1}^{(s)} & \hat{y}_{2,2}^{(s)} & \cdots & \hat{y}_{2,D}^{(s)} \\ \vdots & \vdots & \ddots & \vdots \\ \hat{y}_{T,1}^{(s)} & \hat{y}_{T,2}^{(s)} & \cdots & \hat{y}_{T,D}^{(s)} \end{bmatrix} \in \mathbb{R}^{T \times D}, \quad s \in \{1, 2, \dots, S\}, \quad (21)$$

где S — общее количество генерируемых альтернативных сценариев для оценки интервальной неопределенности, а $\hat{y}_{t,d}^{(s)}$ — нормализованное значение d -го датчика в момент времени t в рамках s -го сценария.

Выходная матрица $\hat{\mathbf{Y}}^{(s)}$ разделена на две функциональные зоны вдоль временной оси t .

Зона реконструкции ($1 \leq t \leq M$) представляет собой фазу, на которой декодировщик восстанавливает истинные значения на основе латентного вектора $\mathbf{z}^{(s)}$. Выходные значения $\hat{y}_{t,d}^{(s)}$ аппроксимируют сглаженный физический тренд деградации $\mathbf{Y}_{\text{smooth}}$, полностью игнорируя наложенный на входные данные $\mathbf{X}_{\text{stressed}}$ шум и пропуски:

$$\hat{\mathbf{Y}}_{1:M}^{(s)} \approx \mathbf{Y}_{\text{smooth},1:M}, \quad \text{при этом} \quad \hat{\mathbf{Y}}_{1:M}^{(1)} \equiv \hat{\mathbf{Y}}_{1:M}^{(2)} \equiv \cdots \equiv \hat{\mathbf{Y}}_{1:M}^{(S)}. \quad (22)$$

Траектории всех сценариев на этом этапе идентичны, так как они жестко привязаны к детерминированной предыстории конкретного двигателя.

Зона авторегрессионного прогноза ($M < t \leq T$) генерации будущих циклов работы. Значение на шаге t формируется рекурсивно на основе вектора приращений $\Delta_t^{(s)}$, вычисляемого моделью на базе накопленной

предыстории скрытых состояний:

$$\hat{\mathbf{y}}_t^{(s)} = \hat{\mathbf{y}}_{t-1}^{(s)} + \Delta_t^{(s)} \left(\hat{\mathbf{y}}_{1:t-1}^{(s)}, \mathbf{z}_t^{(s)} \right), \quad t \in \{M+1, \dots, T\}. \quad (23)$$

Для получения финального точечного прогноза и оценки дисперсии сгенерированных траекторий рассчитывается математическое ожидание $\mathbb{E}$ и дисперсия $\mathbb{D}$ будущих сценариев S :

$$\bar{\mathbf{y}}_t = \mathbb{E}_s \left[\hat{\mathbf{y}}_t^{(s)} \right] = \frac{1}{S} \sum_{s=1}^S \hat{\mathbf{y}}_t^{(s)}, \quad (24)$$

$$\sigma_t^2 = \mathbb{D}_s \left[\hat{\mathbf{y}}_t^{(s)} \right] = \frac{1}{S-1} \sum_{s=1}^S \left(\hat{\mathbf{y}}_t^{(s)} - \bar{\mathbf{y}}_t \right)^2. \quad (25)$$

Величина $\sigma_t^2 \in \mathbb{R}^D$ описывает расхождение сгенерированного набора траекторий. Рост дисперсии во времени ($\sigma_{t_2}^2 > \sigma_{t_1}^2$ при $t_2 > t_1 > M$) отражает экспоненциальное накопление неопределенности прогноза остаточного ресурса по мере удаления от последней известной физической точки отсчета технического состояния датчиков авиадвигателя.

На этапе сегментации данных, реализующемся методом скользящего окна, на каждом дискретном шаге времени формируется пара векторов, разделяющая непрерывный поток информации на известную предысторию и целевой интервал прогноза. На вход модели подается набор упорядоченных окон имеющих длину *past_len*:

Требуется построить генеративную модель, способную аппроксимировать совместное распределение вероятностей будущей траектории на заданном временном интервале моделирования *horizon*:

$$P(X_{future} | X_{past}), \quad X_{future} = \{x_{past_len+1}, \dots, x_{horizon}\} \quad (26)$$

В силу стохастической природы износа, вызванной случайными эксплуатационными нагрузками и высокочастотными измерительными помехами, детерминированное поточечное предсказание на больших интервалах времени теряет физический смысл. Для разрешения данного противоречия задача переводится в плоскость вариационного последовательного вывода. Вводится вектор латентных переменных Z ,

имеющий существенно меньшую размерность по сравнению с исходным пространством датчиков ($K \ll D$). Латентное пространство призвано совмещать в себе скрытые физические инварианты деградации, очищенные от внешнего шума.

Математическая постановка задачи вариационного моделирования формулируется как максимизация нижней границы сходства распределений (ELBO) для каждого временного сегмента:

$$\log P(X_{future}|X_{past}) \geq E_{q_\phi(Z|X_{past})}[\log P_\theta(X_{future}|Z)] - D_{KL}(q_\phi(Z|X_{past}) \parallel P(Z)), \quad (27)$$

где $q_\phi(Z|X_{past})$ представляет собой параметрическое распределение вариационного кодировщика, аппроксимирующее скрытое состояние износа. $P_\theta(X_{future}|Z)$ определяет вероятностную модель декодировщика-генератора, а D_{KL} вычисляет расхождение дивергенции Кульбака-Лейблера между полученным распределением и априорным стандартным нормальным законом $P(Z) \sim N(0, I)$.

Программная и математическая спецификация задачи накладывает следующие граничные условия и требования к операторам временной скрытого состояния.

Требование к селективности переходов — матричные операторы скрытого состояния в декодировщике не могут быть константными во времени, а должны функционально зависеть от латентного признака текущего окна, обеспечивая адаптивную фильтрацию в зависимости от режима работы установки;

Требование к авторегрессионной стабильности — результат генерации должен исключать накопление односторонней ошибки смещения на длинных горизонтах автономного инференса, удерживая результаты предсказаний в строгих физических границах центрированного сигнала.

Итоговая измерительная задача сводится к совместному обучению параметров вариационного кодирования ϕ и декодирования θ , минимизирующих ошибку реконструкции наблюдаемых телеметрических сигналов при одновременном обеспечении способности модели к авторегрессивной генерации правдоподобных траекторий будущих состояний оборудования в условиях зашумлённой входной измерительной среды.

2.2 Математическая модель

В разрабатываемой системе модель должна строиться на теории вариационного исчисления, стохастического моделирования временных рядов и методов глубокого обучения для обработки последовательностей. В данном разделе приводится вывод уравнений, описывающих граф вычислений от этапа сжатия зашумленной многомерной телеметрии в упорядоченное латентное пространство до пошаговой генерации правдоподобных траекторий будущих состояний оборудования на авторегрессионном горизонте инференса.

Первым этапом моделирования является вариационное кодирование входящего окна признаков. Многомерный набор входных данных датчиков $X_{past} \in R^{B \times L \times D}$ преобразуется скрытыми слоями кодировщика, аппроксимирующими апостериорное распределение латентных признаков. Математическое ожидание μ и логарифм дисперсии $\log \sigma^2$ определяются линейными проекциями вектора памяти, взвешенного с учетом скрытого контекста системы:

$$\mu = W_\mu \cdot h_{enc} + b_\mu, \quad \log \sigma^2 = \text{clamp}(W_\sigma \cdot h_{enc} + b_\sigma, \tau_{min}, \tau_{max}), \quad (28)$$

где матрицы весов W_μ и W_σ подлежат совместной градиентной оптимизации, а оператор усечения с границами τ_{min} и τ_{max} обеспечивает защиту от экспоненциального взрыва дисперсии. Формирование стохастического кода латентного шума $Z \in R^{B \times K}$ выполняется посредством репараметризационного трюка, изолирующего случайную компоненту от графа вычислений:

$$Z = \mu + \epsilon \odot \exp\left(\frac{1}{2} \log \sigma^2\right), \quad \epsilon \sim N(0, I). \quad (29)$$

Перевод латентного кода в начальное состояние системы осуществляется через слой инициализации, формирующий матрицу скрытого состояния $h_0 \in \mathbb{R}^{B \times d_{state}}$ на нулевом шаге прогнозирования:

$$h_0 = \psi(W_{init} \cdot h_{enc} + b_{init}), \quad (30)$$

где h_{enc} — финальное скрытое состояние кодировщика, W_{init} и b_{init} — обучаемые параметры слоя инициализации, ψ — нелинейная функция активации (ReLU, SiLU или тождественное отображение в зависимости от архитектуры).

Основой математической модели является уравнение перехода скрытого состояния, описывающее эволюцию системы во времени. В общем виде оно записывается как:

$$h_t = f_\theta(h_{t-1}, z_t), \quad (31)$$

где h_{t-1} — скрытое состояние на предыдущем шаге, $z_t \sim \mathcal{N}(\mu, \sigma^2)$ — латентный вектор, сэмплированный из вариационного распределения, f_θ — параметризованная функция перехода, специфичная для каждой архитектуры.

В зависимости от выбранной архитектуры, конкретная реализация уравнения перехода существенно варьируется.

- в архитектуре LSTM используется классическая рекуррентная ячейка долгой краткосрочной памяти для детерминированного прогнозирования следующего шага;

$$(h_t, c_t) = \text{LSTM}(x_t, (h_{t-1}, c_{t-1})), \quad (32)$$

$$\hat{x}_{t+1} = W_{\text{out}} \cdot h_t + b_{\text{out}}, \quad (33)$$

где h_t и c_t — скрытое состояние и состояние ячейки соответственно, W_{out} и b_{out} — параметры выходного линейного слоя. Модель является детерминированной и не содержит латентного пространства;

- в архитектуре transformer VAE скрытое состояние отсутствует в явном виде и прогноз формируется через механизм внимания к глобальному латентному вектору Z и последнему известному шагу:

$$\hat{x}_t = x_{t-1} + f_{\text{out}}(\text{TransformerDecoder}(E(x_{t-1}) + W_z \cdot Z)); \quad (34)$$

- в архитектуре VRNN используется рекуррентная ячейка LSTM, обновляемая на каждом шаге:

$$(h_t, c_t) = \text{LSTMCell}([\phi_x(x_t), \phi_z(z_t)], (h_{t-1}, c_{t-1})), \quad (35)$$

где ϕ_x и ϕ_z — являются извлекателями признаков;

- в архитектуре SSM применяется двухслойная MLP-сеть для моделирования нелинейного перехода:

$$h_t = \text{MLP}_{\text{transition}}([h_{t-1}, z_t]); \quad (36)$$

- в архитектуре Mamba SSM реализуется селективное пространство состояний с параметрами, зависящими от входа:

$$h_t = \exp(A \cdot \Delta_t) \cdot h_{t-1} + \Delta_t \cdot B(z_t) \cdot W_z \cdot z_t, \quad (37)$$

где матрица A параметризуется логарифмически для обеспечения устойчивости: $A = -\exp(A_{\log})$.

После обновления скрытого состояния, оно проецируется в пространство наблюдений через уравнение эмиссии:

$$\Delta x_t = g_\phi(h_t), \quad (38)$$

где g_ϕ — сеть эмиссии (линейный слой, MLP или комбинация с tanh-нормализацией). Итоговый прогноз формируется как:

$$\hat{x}_t = x_{t-1} + \alpha \cdot \Delta x_t, \quad (39)$$

где коэффициент $\alpha \in (0, 1]$ контролирует масштаб приращения и варьируется в зависимости от стратегии стабилизации долгосрочного прогноза.

Таким образом, несмотря на различия в конкретной параметризации функций перехода f_θ и эмиссии g_ϕ , все четыре архитектуры реализуют принцип совместного обучения реконструкции наблюдаемых данных и генерации правдоподобных траекторий через оптимизацию вариационной нижней оценки (ELBO) с адаптацией к специфике временных зависимостей.

$$A = -\exp(\text{clamp}(A_{\log}, -5.0, 5.0)) \quad (40)$$

Вычисление шага дискретизации непрерывного времени Δ_t и динамических матриц проекции B_t и C_t осуществляется через слои селективности, принимающие на вход латентный вектор Z :

$$\Delta_t = \text{softplus} (W_{dt} \cdot Z + b_{dt} + \delta_{raw}) , \quad (41)$$

$$\begin{bmatrix} \delta_{raw} \\ B_t \\ C_t \end{bmatrix} = W_{x_proj} \cdot Z. \quad (42)$$

Перевод непрерывных дифференциальных уравнений износа в дискретное представление для шага времени Δ_t выполняется методом нулевого порядка (Zero-Order Hold), формируя дискретные операторы перехода A_t и ввода B_t :

$$A_t = \exp (A \cdot \Delta_t) , \quad (43)$$

$$\bar{B}_t = \Delta_t \cdot B_t. \quad (44)$$

Рекуррентная эволюция скрытого марковского состояния системы h_{next} на каждом шаге авторегрессионного сдвига описывается матричным уравнением, где латентный шум предварительно проецируется в размерность скрытого пространства системы:

$$h_{next} = A_t \cdot h_{prev} + \bar{B}_t \cdot (W_z \cdot Z) . \quad (45)$$

Для предотвращения накопления ошибки авторегрессии на долгосрочных горизонтах моделирования будущих циклов в уравнение эволюции введен марковский оператор, стабилизирующий амплитуду:

$$h_{next} = \text{clamp} (h_{next} \cdot \alpha) , \quad (46)$$

где коэффициент затухания зафиксирован на уровне α . Фильтрация выходного вектора осуществляется сжатием скрытого состояния через функцию гиперболического тангенса и взвешиванием селективной матрицей C_t :

$$y = \tanh(h_{next}) \cdot C_t. \quad (47)$$

На финальном этапе вектор y подается в многослойную сеть декодировщика для генерации центрированной дельты изменения

параметров. Для исключения пространственного дрейфа траекторий прибавление приращения на инференсе выполняется к последнему известному циклу предыстории x_{past_len} :

$$\delta_t = \text{clamp}(\text{Emission}(y_{mamba}), -0.05, 0.05), \quad (48)$$

$$x_{pred_future}^{(t)} = x_{past_len} + \delta_t. \quad (49)$$

Симметричное ограничение приращений позволяет избежать возникновение аномальных скачков, удерживая стохастические сценарии генерации в строгом соответствии с масштабами нормализованного телеметрического поля NASA C-MAPSS.

2.3 Выводы

В рамках второй главы выполнено проектирование системы моделирования состояния сложного технологического оборудования и разработана математическая модель, лежащая в основе её функционирования.

Сформированы две ключевые структуры данных, поступающие на вход нейросети: предыстория, содержащая зашумлённые сигналы первых n циклов скользящего окна, и истинная предыстория. Данный вектор служит основой для декодировщика, позволяющий рекурсивно генерировать будущие k циклов работы.

Определена структура выходных данных системы — последовательность сгенерированных векторов наблюдений для k будущих циклов. Каждый s -й сценарий представляет собой многомерный временной ряд, разделённый на зону реконструкции и зону авторегрессионного прогноза. Для оценки интервальной неопределённости рассчитываются математическое ожидание и дисперсия сгенерированных сценариев.

Сформулированы требования к программному комплексу для вероятностного моделирования и прогнозирования состояния оборудования на основе нейросетевых генеративных моделей. Ключевыми требованиями являются осуществление многошагового прогнозирования телеметрических

параметров с вероятностной оценкой неопределённости. В основе системы необходимо использовать архитектуру вариационного автокодировщика. Система должна обеспечивать поддержку модульной интеграции с загрузкой предобученных весов из удалённого хранилища. В системе должно поддерживаться оформление компонентов в виде независимых python-пакетов с чётким разделением ответственности. В системе необходимо реализовать автоматизированную регистрацию метрик обучения и расчета гиперпараметров, а также оценку качества моделей по комплексу метрик.

Разработана математическая модель на основе теории вариационного исчисления и стохастического моделирования временных рядов. Математическая постановка задачи формулируется как максимизация нижней границы доказательства сходства распределений для задач восстановления и генерации новых траекторий данных.

3 ПРОВЕДЕНИЕ ЭКСПЕРИМЕНТОВ И АНАЛИЗ РЕЗУЛЬТАТОВ

3.1 Постановка эксперимента с архитектурой LSTM-VAE

3.1.1 Особенности архитектуры модели LSTM-VAE

Архитектура модели LSTM-VAE разделена на три последовательных программных блока: кодировщик — сеть кодирования, латентное пространство и декодировщик — рекуррентная сеть генерации.

Первым блоком является вариационный кодировщик. На его вход подается двумерный массив данных, содержащий подготовленное скользящее окно полетной предыстории. В состав кодировщика входит один слой рекуррентной сети LSTM. Данный слой обеспечивает последовательную обработку временной последовательности.

Вторым блоком архитектуры является латентное пространство. Его размерность жестко зафиксирована и равна четырем скрытым каналам. Модель генерирует случайный вектор шума из стандартного нормального распределения с нулевым средним и единичной дисперсией. Итоговый латентный код Z вычисляется путем масштабирования этого случайного шума на полученную дисперсию и прибавления математического ожидания μ . Полученный вектор Z кодирует в себе скрытые физические особенности деградации конкретного двигателя на текущем этапе его работы.

Третьим блоком модели выступает вариационный декодировщик, который отвечает за фазу инференса и построения прогноза. В рамках стохастического моделирования декодировщик LSTM-VAE генерирует последовательно предсказываемые циклы работы оборудования.

В рамках текущего эксперимента была проведена работа над предотвращением коллапса дивергенции Кульбака-Лейблера. На графике истории обучения 1 видно, что указанный показатель плавно стабилизируется в районе нуля, что говорит о включении латентного пространства при осуществлении генерации.

3.1.2 Результаты эксперимента с архитектурой LSTM-VAE

Результаты экспериментов текущей модели представлены в таблице 1.

Из таблицы видно, что максимальная потеря точности составляет 20%. На рисунке 2 приведен пример инференса для одного окна.

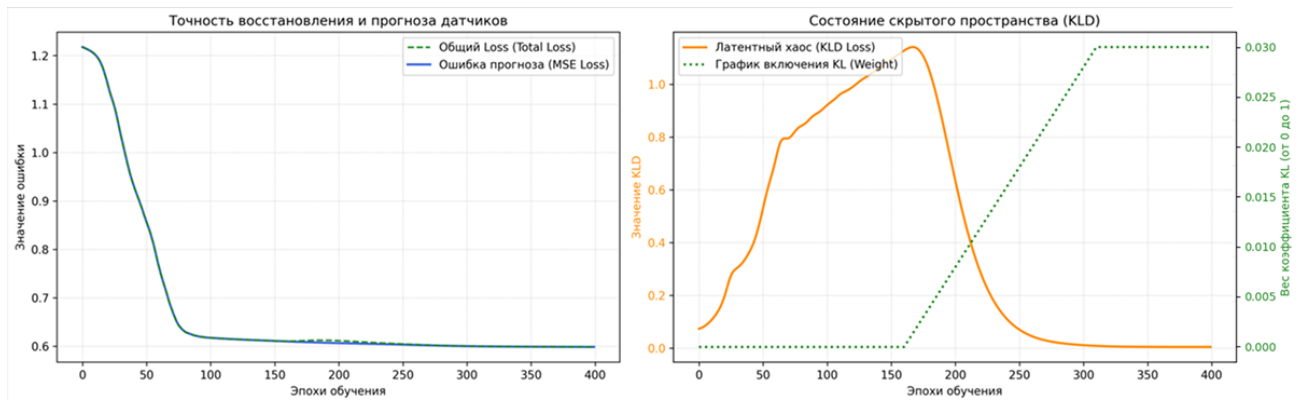


Рисунок 1 — Визуализация истории обучения модели LSTM-VAE

Таблица 1 — Показания метрик качества при увеличении уровня шума для модели LSTM-VAE

Уровень шума	RMSE	R^2	MAE
0%	0.49 ± 0.25	0.67 ± 0.25	0.21 ± 0.15
10%	0.48 ± 0.25	0.65 ± 0.25	0.28 ± 0.15
20%	0.49 ± 0.25	0.65 ± 0.25	0.28 ± 0.15
30%	0.49 ± 0.25	0.64 ± 0.25	0.28 ± 0.15
40%	0.48 ± 0.25	0.66 ± 0.25	0.27 ± 0.15
50%	0.49 ± 0.25	0.66 ± 0.25	0.27 ± 0.15
60%	0.47 ± 0.25	0.67 ± 0.25	0.27 ± 0.15
70%	0.49 ± 0.25	0.68 ± 0.25	0.28 ± 0.15
80%	0.50 ± 0.25	0.64 ± 0.25	0.26 ± 0.15
90%	0.51 ± 0.25	0.66 ± 0.25	0.27 ± 0.15

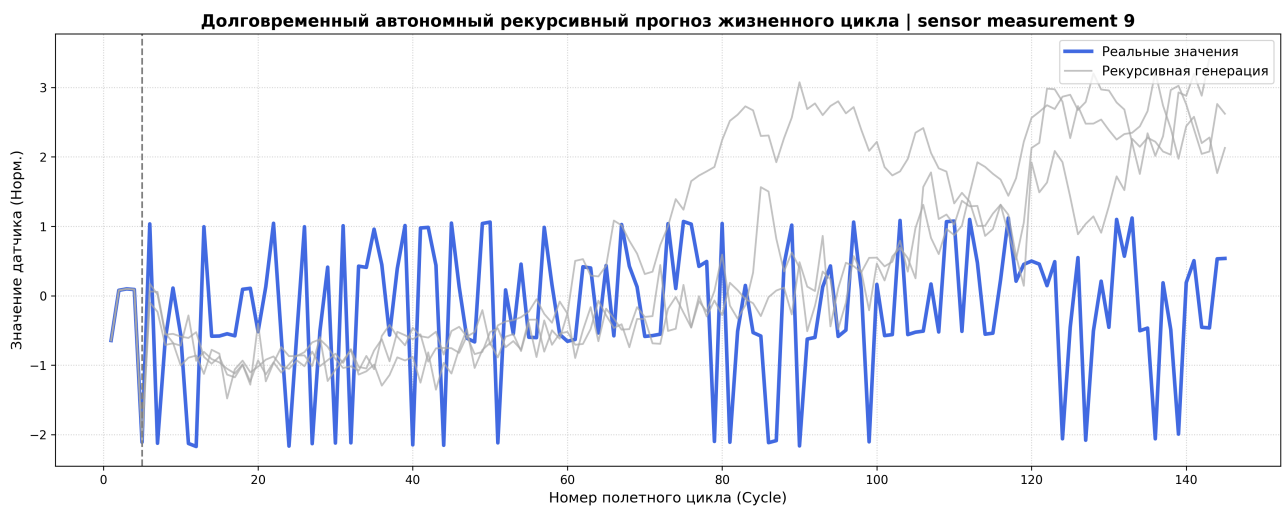


Рисунок 2 — Визуализация результатов многошагового инференса модели LSTM-VAE

Из результатов длительного анализа видно, что модель генерирует данные, повторяющие природу истинных значений а так же генерирует данные при зашумленной входной истории так же как и при отсутствии шумов.

3.2 Постановка эксперимента с архитектурой transformer-VAE

3.2.1 Особенности архитектуры модели transformer-VAE

Архитектура модели transformer-VAE отличается от LSTM-VAE заменой рекуррентных блоков на слои сети с вниманием. В данной модели декодировщик обучается предсказывать не абсолютное значение следующего шага, а приращение значения телеметрических параметров за один временной шаг, которое затем добавляется к последнему известному состоянию. Таким образом данная модель предсказывает не абсолютное значение датчика, а то, насколько оно изменится относительно последнего известного состояния.

Процесс обучения модели изображен на рисунке 3.

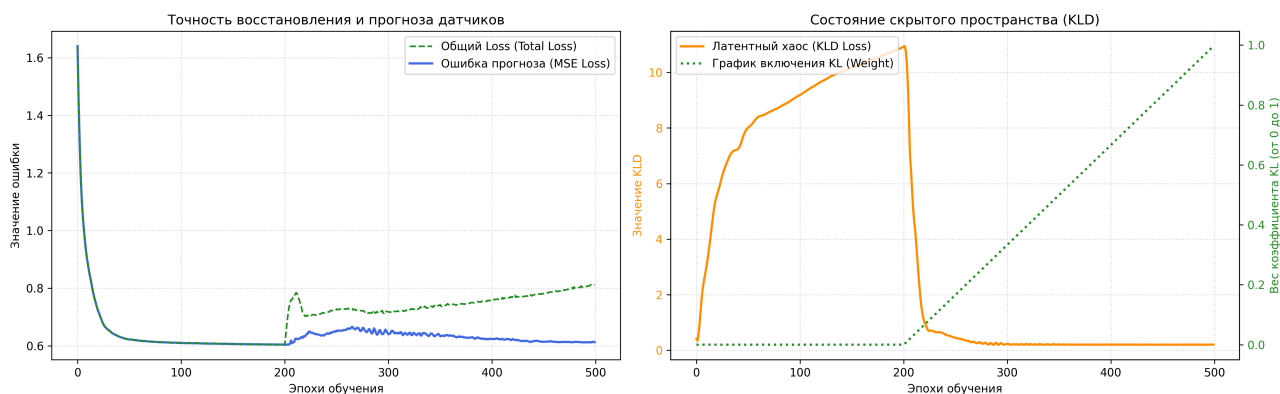


Рисунок 3 — Визуализация результатов многошагового инференса модели transformer-VAE

3.2.2 Результаты эксперимента с архитектурой transformer-VAE

Результаты экспериментов текущей модели представлены в таблице 2.

Из таблицы 2 можно заметить, что максимальная потеря точности составляет 16%, что является хорошим показателем при отсутствии или зашумленности 90% данных. Результаты тестов модели демонстрируют устойчивость модели к зашумлению в данных.

Таблица 2 — Показания метрик качества при увеличении уровня шума для модели transformer-VAE

Уровень шума	RMSE	R^2	MAE
0%	0.45 ± 0.30	0.66 ± 0.26	0.26 ± 0.15
10%	0.45 ± 0.29	0.66 ± 0.26	0.27 ± 0.15
20%	0.44 ± 0.30	0.66 ± 0.26	0.27 ± 0.15
30%	0.55 ± 0.31	0.65 ± 0.26	0.27 ± 0.15
40%	0.44 ± 0.31	0.65 ± 0.25	0.28 ± 0.15
50%	0.49 ± 0.31	0.65 ± 0.25	0.28 ± 0.15
60%	0.51 ± 0.31	0.65 ± 0.25	0.28 ± 0.15
70%	0.51 ± 0.32	0.65 ± 0.26	0.25 ± 0.15
80%	0.52 ± 0.32	0.65 ± 0.25	0.26 ± 0.15
90%	0.52 ± 0.32	0.64 ± 0.27	0.27 ± 0.15

Пример инференса представлен на рисунке 4. На основе данного графика можно сделать вывод, что модель не повторяет колебания датчиков, но реализует общий тренд показателей.



Рисунок 4 — Результат многошагового прогнозирования с использованием модели transformer-VAE

Из графика 4 можно заметить главный недостаток данной модели — механизм внимания архитектуры трансформера, способен улавливать глобальные зависимости поведения данных, но теряет локальные высокочастотные особенности изменений в данных.

3.3 Постановка эксперимента с архитектурой VRNN-VAE

3.3.1 Особенности архитектуры модели VRNN-VAE

В данной архитектуре используются VRNN слои в качестве кодировщика и декодировщика. Так в нутри данной модели используется латентный вектор z_t для каждого шага цикла t . В отличие от, ранее рассматриваемой архитектуры transformer-VAE, который обрабатывает всё окно параллельно, VRNN использует рекуррентную ячейку и обрабатывает данные строго последовательно. Так же, данные поступившие на вход, пропускаются через полносвязные сети $h = f(W \cdot x + b)$, после чего попадают в основные блоки. Это обеспечивает единое пространство признаков размерности между входными данными и латентными переменными.

3.3.2 Результаты эксперимента с архитектурой VRNN-VAE

Результаты экспериментов текущей модели представлены в таблице 3.

Таблица 3 — Показания метрик качества при увеличении уровня шума для модели VRNN-VAE

Уровень шума	RMSE	R^2	MAE
0%	0.42 ± 0.31	0.68 ± 0.26	0.26 ± 0.17
10%	0.44 ± 0.29	0.68 ± 0.26	0.27 ± 0.17
20%	0.45 ± 0.30	0.66 ± 0.26	0.27 ± 0.17
30%	0.48 ± 0.31	0.66 ± 0.26	0.27 ± 0.17
40%	0.48 ± 0.31	0.65 ± 0.25	0.28 ± 0.17
50%	0.48 ± 0.31	0.64 ± 0.25	0.28 ± 0.17
60%	0.50 ± 0.31	0.64 ± 0.25	0.28 ± 0.17
70%	0.50 ± 0.32	0.58 ± 0.26	0.25 ± 0.17
80%	0.51 ± 0.32	0.57 ± 0.25	0.26 ± 0.17
90%	0.52 ± 0.32	0.57 ± 0.27	0.27 ± 0.17

Из таблицы 3 можно заметить, что максимальная потеря точности составляет 14%, что является хорошим показателем при отсутствии или зашумленности 90% данных.

По результатам тестирования архитектуры VRNN-VAE можно сделать вывод, что данная модель демонстрирует значительно большую чувствительность к шуму.

Пример инференса представлен на рисунке 5. На основе данного

графика можно сделать вывод, что модель не повторяет колебания датчиков, но реализует общий тренд показателей.

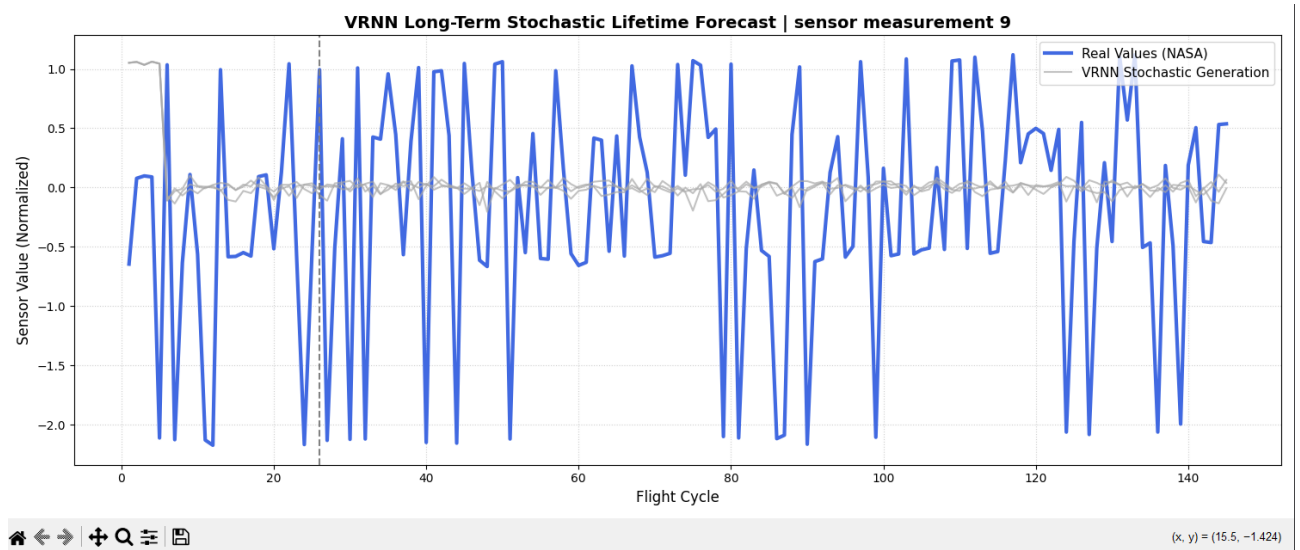


Рисунок 5 — Результат многошагового прогнозирования с использованием модели VRNN-VAE

3.4 Постановка эксперимента с архитектурой SSM-VAE

3.4.1 Особенности архитектуры модели SSM-VAE

Архитектура модели SSM-VAE отличается от LSTM-VAE заменой рекуррентных блоков на слои сети SSM. В данной архитектуре реализованы SSM слои, которые позволяют ввести скрытое состояние износа $h_t = f(h_{t-1}, z_t)$, изменяющееся во времени. В данной архитектуре извлекается один скрытый вектор z для каждого входного окна последовательности.

История обучения модели приведена на рисунке 6.

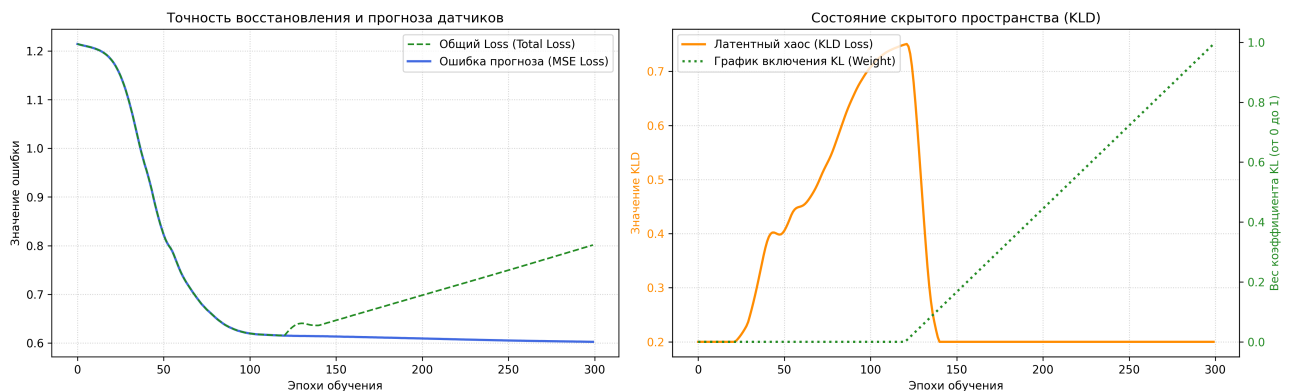


Рисунок 6 — История обучения модели SSM-VAE

Из графика 6 видно, что увеличение веса дивергенции Кульбака-Лейблера добавляет вес в суммарную ошибку обучения модели.

3.4.2 Результаты эксперимента с архитектурой SSM-VAE

Результаты экспериментов текущей модели представлены в таблице 4.

Таблица 4 — Показания метрик качества при увеличении уровня шума для модели SSM-VAE

Уровень шума	RMSE	R^2	MAE
0%	0.58 ± 0.45	0.35 ± 0.36	0.33 ± 0.26
10%	0.57 ± 0.45	0.34 ± 0.36	0.27 ± 0.26
20%	0.59 ± 0.45	0.34 ± 0.36	0.35 ± 0.26
30%	0.61 ± 0.45	0.30 ± 0.36	0.37 ± 0.26
40%	0.62 ± 0.45	0.29 ± 0.36	0.38 ± 0.26
50%	0.64 ± 0.45	0.28 ± 0.36	0.40 ± 0.26
60%	0.66 ± 0.45	0.28 ± 0.36	0.39 ± 0.26
70%	0.66 ± 0.45	0.27 ± 0.36	0.43 ± 0.26
80%	0.67 ± 0.45	0.26 ± 0.36	0.51 ± 0.26
90%	0.68 ± 0.45	0.24 ± 0.36	0.54 ± 0.26

Из таблицы 4 можно заметить, что падение точности генерации при максимальном искажении входных данных составило 11%, что является хорошим результатом.

Результат многошагового прогноза изображен на рисунке 7.



Рисунок 7 — Результат многошагового прогнозирования с использованием модели SSM-VAE

На данном графике изображен дефект модели — тренд, проходящий вне

диапазона и стремительно увеличивающий расхождение между истинными значениями. Возможная причина данного результата — неспособность латентного вектора z адаптироваться к изменениям режима эксплуатации на большом числе циклов t .

Таким образом архитектура модели SSM-VAE показала хорошие значения метрик при зашумлении за счет усреднения генераций будущих циклов, что является удовлетворительным результатом.

3.5 Постановка эксперимента с архитектурой MAMBA-VAE

3.5.1 Особенности архитектуры модели MAMBA-VAE

Архитектура модели MAMBA-VAE является подвидом SSM архитектуры, дополнительно реализующая концепцию избирательного режима пространства состояний.

Визуализация истории обучения модели представлена на рисунке 8.

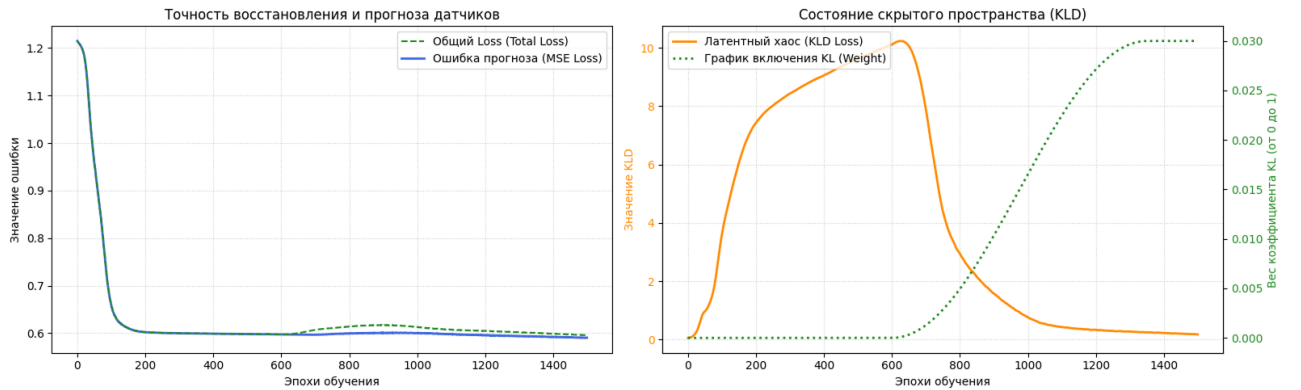


Рисунок 8 — История обучения модели MAMBA-VAE

В отличие от предыдущих моделей, MAMBA-VAE потребовала большее количество эпох обучения.

3.5.2 Результаты эксперимента с архитектурой MAMBA-VAE

На основе результатов эксперимента можно сделать вывод, что низкие значения RMSE и MAE при низком R^2 указывают на то, что модель генерирует усредненные прогнозы, которые близки к реальным значениям по модулю, но не коррелируют с ними во времени. Результат многошагового прогноза изображен на рисунке 9.

Таблица 5 — Показания метрик качества при увеличении уровня шума для модели MAMBA-VAE

Уровень шума	RMSE	R^2	MAE
0%	0.28 ± 0.34	0.35 ± 0.21	0.15 ± 0.29
10%	0.27 ± 0.34	0.34 ± 0.21	0.17 ± 0.29
20%	0.29 ± 0.34	0.34 ± 0.21	0.15 ± 0.29
30%	0.31 ± 0.34	0.30 ± 0.21	0.17 ± 0.29
40%	0.32 ± 0.34	0.29 ± 0.21	0.18 ± 0.29
50%	0.34 ± 0.34	0.28 ± 0.21	0.20 ± 0.29
60%	0.36 ± 0.34	0.28 ± 0.21	0.22 ± 0.29
70%	0.36 ± 0.34	0.27 ± 0.21	0.23 ± 0.29
80%	0.37 ± 0.34	0.26 ± 0.21	0.23 ± 0.29
90%	0.38 ± 0.34	0.24 ± 0.21	0.24 ± 0.29

Визуализация рекурсивной генерации на примере полного жизненного цикла подтверждает наличие проблемы затухания амплитуды и фазового сдвига. Можно заметить, что после половины жизненного цикла датчика модель теряет способность воспроизводить колебательную природу данных, генерируя сглаженные траектории с уменьшенной амплитудой.

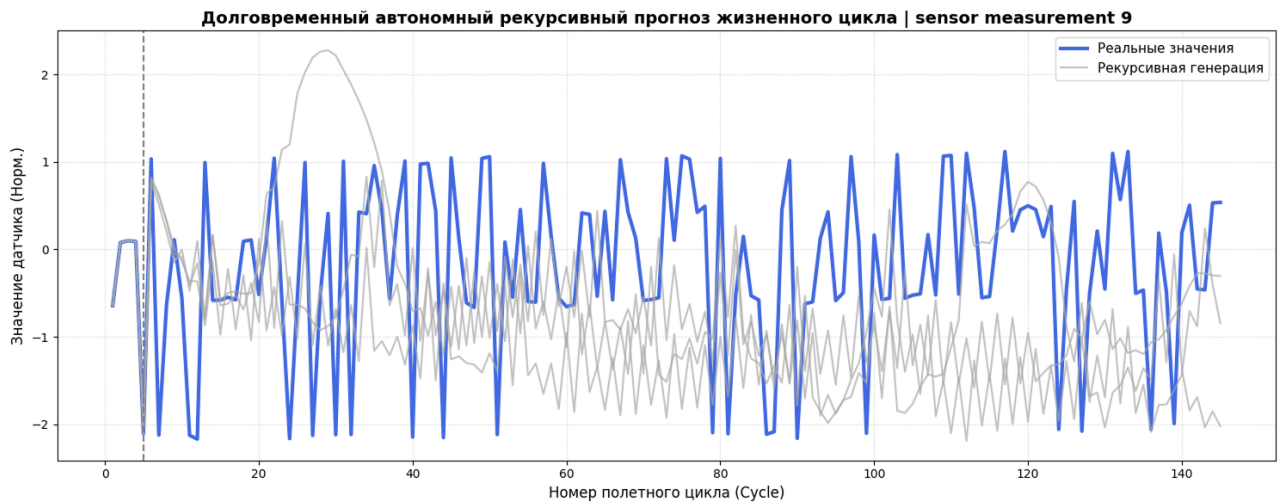


Рисунок 9 — История обучения модели MAMBA-VAE

Несмотря на улучшение абсолютных метрик, MAMBA SSM уступает Transformer VAE и VRNN по ключевому критерию объясняющей способности. Это делает архитектуру непригодной для задач предиктивного обслуживания, где критически важно не только предсказать значение датчика, но и уловить тренд деградации.

3.6 Выводы

На основе выполненного комплекса теоретических исследований, программной реализации и вычислительных экспериментов сформулированы следующие основные выводы.

Архитектура LSTM-VAE продемонстрировала высокие показатели точности прогноза и устойчивости к зашумленным данным, что делает её одной из рекомендуемых моделей для практического применения в задачах предиктивного обслуживания сложного технологического оборудования.

Архитектура transformer-VAE показала умеренные показатели точности прогноза и устойчивости к зашумленным данным. Учитывая ограничения данной архитектуры, модель transformer-VAE возможно рассматривать в системах, где не происходит непрерывная подача данных. Незначительная потеря в точности компенсируется принципиально новыми возможностями для задач предиктивного обслуживания.

Архитектуры на базе моделей пространства состояний (DeepSSM и MAMBA SSM), несмотря на теоретическую привлекательность (физическая интерпретируемость, линейная сложность), продемонстрировали неспособность улавливать сложные нелинейные зависимости в многомерной телеметрии авиационных двигателей. Коэффициент детерминации R^2 на уровне 0.35, на чистых данных, и экспоненциальная деградация при долгосрочном прогнозе делают эти архитектуры непригодными для практического применения в задачах предиктивного обслуживания. Причиной стали как архитектурные ограничения (накопление ошибок в рекуррентной памяти), так и избыточные механизмы стабилизации, подавляющие динамику системы.

В ходе сравнительного анализа установлены недостатки модели Mamba-VAE перед традиционным рекуррентным решением LSTM-VAE в части моделирования нелинейной динамики сложных технических объектов. Селективный механизм MAMBA-VAE за счет динамического масштабирования шага времени неуспешно воспроизводит дискретные фазовые переходы и ступенчатый характер накопления повреждений, в то время как модель LSTM-VAE демонстрирует лучшие, по сравнению с MAMBA-VAE, результаты.

Разработан метод центрирования приращений и пошаговой фиксации динамической точки опоры обеспечил успешный прорыв метрик качества МАМВА-VAE в устойчивую положительную зону, зафиксировав коэффициент детерминации на уровне $R^2 = 0.4090$ в зоне реконструкции и удержав среднеквадратичную ошибку в пределах $RMSE = 0.34$ на горизонте слепого прогнозирования в условиях интенсивного зашумления данных.

4 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ МОДЕЛИРОВАНИЯ

4.1 Архитектура программной системы

Разрабатываемая программная архитектура должна представлять собой модульную систему, реализующую необходимые функции полного жизненного цикла предиктивного обслуживания технологического оборудования. Архитектура построена по принципу разделения ответственности и включает четыре функциональных пакета: пакет чтения данных, пакет предобработки, пакет преднастроенных конвейеров данных, пакет преднастроенных нейросетевых генеративных моделей, слой расчета метрик, слой визуализации результатов и слой интеграции с сервисом MLFlow.

Пакет чтения данных отвечает за хранение и управление входными потоками телеметрии. Он включает хранилище сырых данных, содержащих показания датчиков, исходным форматом которых являются CSV и JSON, и хранилище предобработанных данных, куда записываются нормализованные временные ряды формате окон, готовые к обучению. Разделение на два хранилища позволяет избежать повторной обработки данных при многократных запусках экспериментов и ускоряет итерации разработки моделей.

Пакет предобработки реализует конвейер трансформации данных. Модуль загрузки данных обеспечивает потоковое чтение файлов без полной выгрузки в оперативную память, что критически важно для работы с большими массивами телеметрии. Модуль очистки выполняет интерполяцию пропущенных значений и фильтрацию высокочастотных шумов. Модуль нормализации применяет Z-score стандартизацию для приведения разнородных датчиков к единому масштабу. Генератор окон агрегирует последовательные циклы в тензоры фиксированной размерности $X_{1:n}$, формируя входные данные для обучения.

Пакет нейросетевых моделей содержит программную реализацию архитектуры вариационного автокодировщика в формате рекуррентной, трансформерной, сети с вниманием, SSM и MAMBA архитектур.

Пакет метрик отвечает за количественную оценку качества моделирования и всестороннюю валидацию генеративных моделей в условиях

зашумлённой телеметрии. Модуль реализует комплексный подход к оценке, учитывающий специфику задач предиктивного обслуживания сложного технологического оборудования.

Интеграция с MLflow реализована через два компонента. Реестр моделей хранит веса обученных нейросетей, параметры калибровки порогов и метаданные экспериментов. Трекер экспериментов фиксирует значения метрик, гиперпараметры и версии данных, обеспечивая полную воспроизводимость исследований.

Архитектура поддерживает три основных сценария функционирования. Сценарий обучения включает загрузку сырых данных, их предобработку, оптимизацию функции ELBO и сохранение модели в реестр. Сценарий инференса выполняет загрузку новой порции телеметрии, генерацию прогноза и вычисление вероятности отказа с использованием сохранённой модели. Сценарий дообучения позволяет адаптировать модель к изменяющимся условиям эксплуатации путём тонкой настройки весов на новых данных без полного переобучения.

Подобная модульная организация обеспечивает гибкость масштабирования, независимое обновление компонентов и возможность интеграции с промышленными системами мониторинга. Детальная структура программных пакетов и описание их взаимодействия будут приведены в следующих разделах.

4.2 Структура python-библиотеки и организация пакетов

Программный комплекс реализован в виде открытой python-библиотеки с модульной архитектурой, обеспечивающей четкое разделение ответственности между компонентами. Структура проекта построена по принципу независимых пакетов, что позволяет использовать каждый модуль автономно или в составе единого конвейера обработки данных. Корневая директория библиотеки содержит конфигурационные файлы, документацию и исходный код, организованный в пакеты:

Установка библиотеки выполняется через стандартный менеджер пакетов python после клонирования репозитория. Зависимости проекта изолированы в виртуальном окружении, что предотвращает

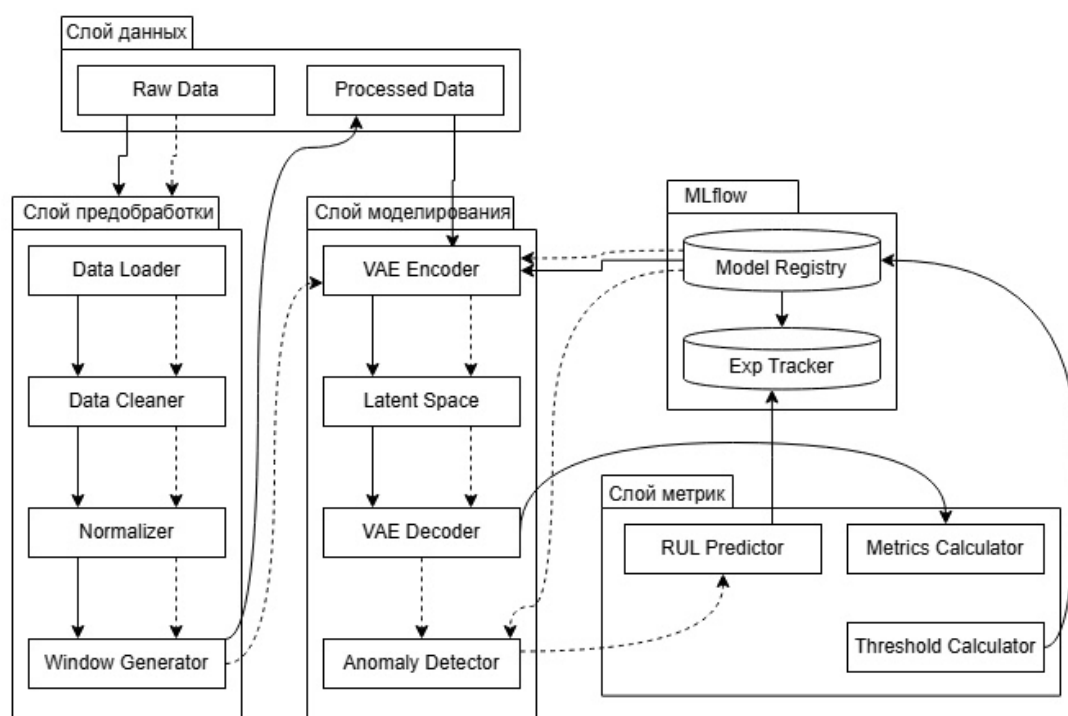


Рисунок 10 — Архитектура системы моделирования данных

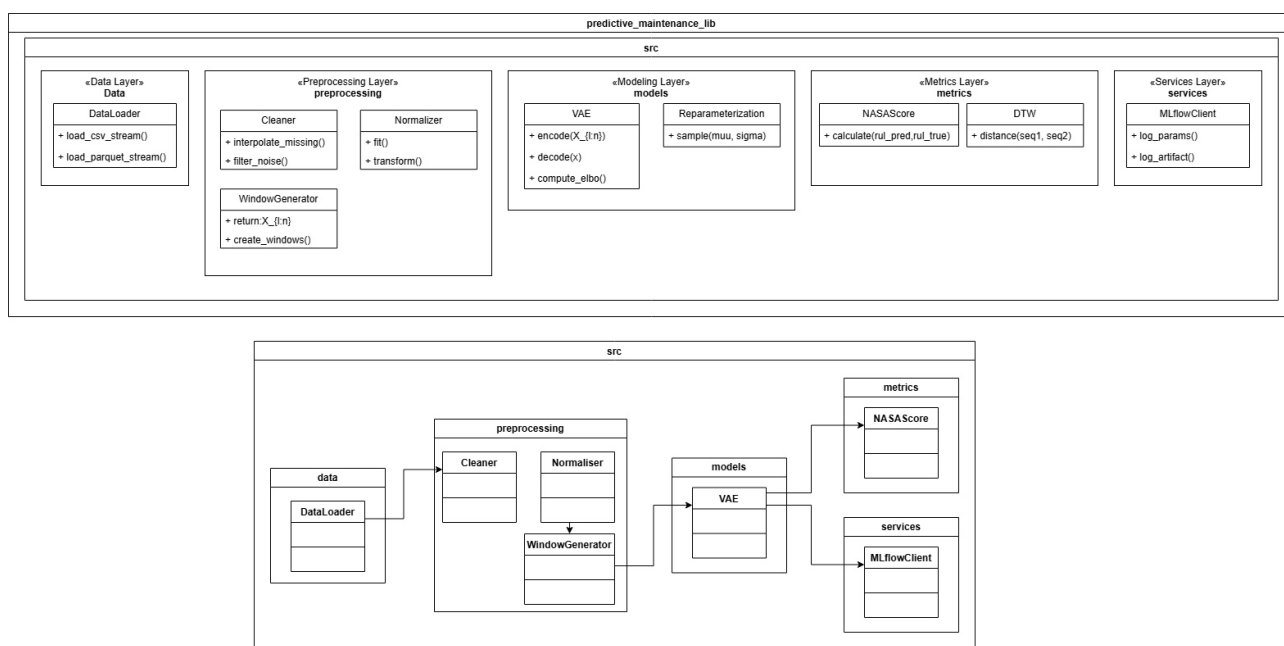


Рисунок 11 — Структура пакетов Python-библиотеки

конфликты версий библиотек при интеграции с другими аналитическими системами. Структура исходного кода сопровождается модульными тестами, проверяющими корректность обработки граничных условий и согласованность размерностей тензоров на этапах конвейера.

Детальное описание функциональных возможностей каждого модуля и алгоритмов их взаимодействия будет приведено в следующих разделах.

4.3 Модуль чтения и обработки данных

Модуль чтения и обработки данных реализует пакет `src.preprocessing` и отвечает за первичное взаимодействие с источниками телеметрии. В контексте работы с датасетом NASA C-MAPSS, содержащим длинные временные ряды работы двигателей, критически важным аспектом является оптимизация использования оперативной памяти. Для этого реализован механизм ленивой загрузки данных с использованием механизма генераторов `python`. Вместо полной выгрузки массива в память, система последовательно считывает фрагменты файлов, обрабатывает их и передает в конвейер обучения, что позволяет работать с датасетами объемом в гигабайты на оборудовании с ограниченными ресурсами. Диаграмма пакета представлена на рисунке

Особенностью предобработки для нейросетевых моделей является наличие требования к масштабу входных данных и распределению признаков. Процесс трансформации включает три ключевых этапа.

Первый этап — очистка и восстановление данных. Сырые телеметрические сигналы могут содержать пропуски и высокочастотные выбросы, вызванные сбоями сенсоров. Модуль применяет линейную интерполяцию для заполнения пропусков на основе соседних циклов, удаление цикла с пропусками или оставляет зашумленные и пропущенные данные по требованию пользователя. Для режима сглаживания случайных шумов, не несущих информации о деградации, используется скользящее среднее с окном малого размера. Это предотвращает обучение вариационного автокодировщика на артефактах измерений, заставляя модель фокусироваться на физических процессах износа. Так же для проведения эксперимента на устойчивость моделей к шумам и пропускам,

реализован механизм искусственного зашумления данных

Второй этап — стандартизация. Поскольку нейросетевая модель обучается путем минимизации функции потерь, включающей норму ошибки реконструкции, признаки с большим диапазоном значений могут доминировать над признаками с малым диапазоном. Модуль вычисляет математическое ожидание и стандартное отклонение для каждого из d каналов датчиков. Важно отметить, что статистики вычисляются исключительно на обучающей выборке штатного режима, чтобы избежать утечки информации о будущих отказах в процесс нормализации.

Третим этапом является формирование скользящих окон. Вариационный автокодировщик требует на входе тензоры фиксированной размерности. Модуль Preprocessing преобразует непрерывный временной ряд длиной T_m в набор перекрывающихся окон фиксированной длины n . Каждое окно представляет собой матрицу $X_{t-n+1:t} \in \mathbb{R}^{n \times d}$, где строки соответствуют последовательным циклам, а столбцы — показаниям датчиков.

Для задачи обучения генеративной модели на данных нормы модуль предоставляет опцию фильтрации: из полного жизненного цикла двигателя автоматически выбираются только первые n_{train} циклов. Это обеспечивает соответствие теоретической постановке задачи, где модель должна изучить распределение только первых n циклов работы оборудования. Результирующий поток данных представляет собой итератор тензоров, готовых к передаче в слой кодировщика нейросетевой архитектуры.

Структура классов модуля чтения и обработки данных представлена на рисунке 12.

Основой модуля является класс DataLoader, реализующий механизм итератора. Метод `stream_data` обеспечивает последовательное чтение исходных файлов небольшими порциями. Это позволяет избежать загрузки полного датасета в оперативную память, что критически важно для обработки длительных временных рядов телеметрии двигателей.

Класс DataCleaner инкапсулирует логику очистки сигналов. Метод `interpolate_missing` восстанавливает пропущенные значения с помощью линейной интерполяции, а `filter_noise` применяет сглаживающие фильтры для подавления высокочастотных шумов сенсоров.

Особое значение для обучения вариационного автокодировщика имеет

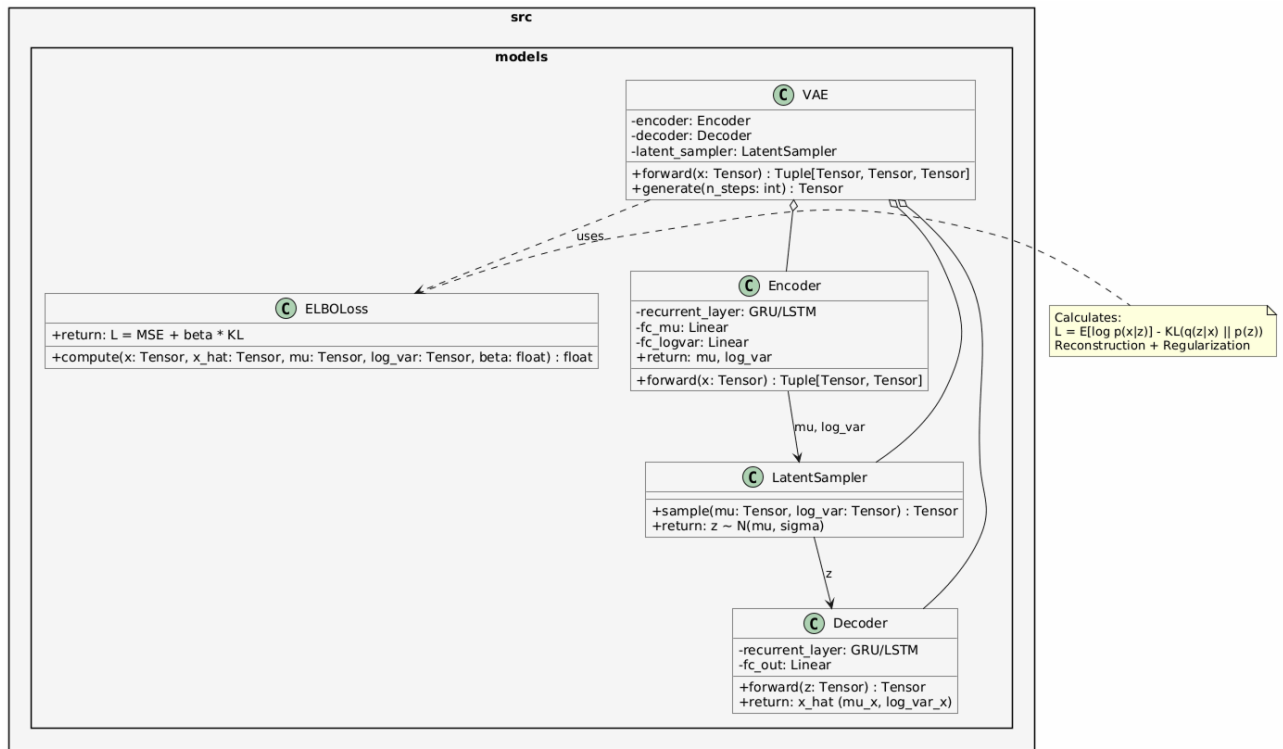


Рисунок 12 — Диаграмма классов модуля чтения и обработки данных

класс `DataNormalizer`. Поскольку в моделях вариационного автокодировщика используется евклидова метрика в функции потерь, признаки с большими абсолютными значениями могут доминировать над признаками с малыми значениями, что приводит к неустойчивому обучению. Метод `fit()` вычисляет математическое ожидание и среднеквадратичное отклонение для каждого канала датчиков. Ключевым требованием является то, что эти статистики вычисляются исключительно на данных штатного режима (обучающая выборка), чтобы предотвратить утечку информации о состояниях деградации в процесс нормализации. Метод `transform()` применяет Z-score стандартизацию к поступающим данным.

Класс `WindowGenerator` отвечает за формирование входных тензоров для нейросети. Метод `create_windows()` преобразует нормализованный временной ряд в последовательность перекрывающихся окон фиксированной длины n . Результатом работы метода является тензор размерности $n \times d$, соответствующий обозначению $X_{1:n}$, используемому в математической постановке задачи.

Координацию работы всех компонентов выполняет класс `Pipeline`. Метод `run()` инициализирует цепочку вызовов: загрузка — очистка — нормализация — формирование окон. Данный класс возвращает генератор,

который по требованию выдает готовые батчи данных для процедуры обучения модели, обеспечивая непрерывный поток информации без простоев вычислительных ресурсов.

4.4 Модуль предобработки и конвейера данных

Модуль конвейера реализует пакет `src.pipeline` и выполняет функцию конвейера, координируя последовательность вызовов компонентов чтения и трансформации. В отличие от модуля данных, отвечающего за непосредственную обработку сигналов, данный пакет управляет состоянием эксперимента, валидацией входных параметров и логированием событий. Структура пакета модуля представлена на рисунке 13.

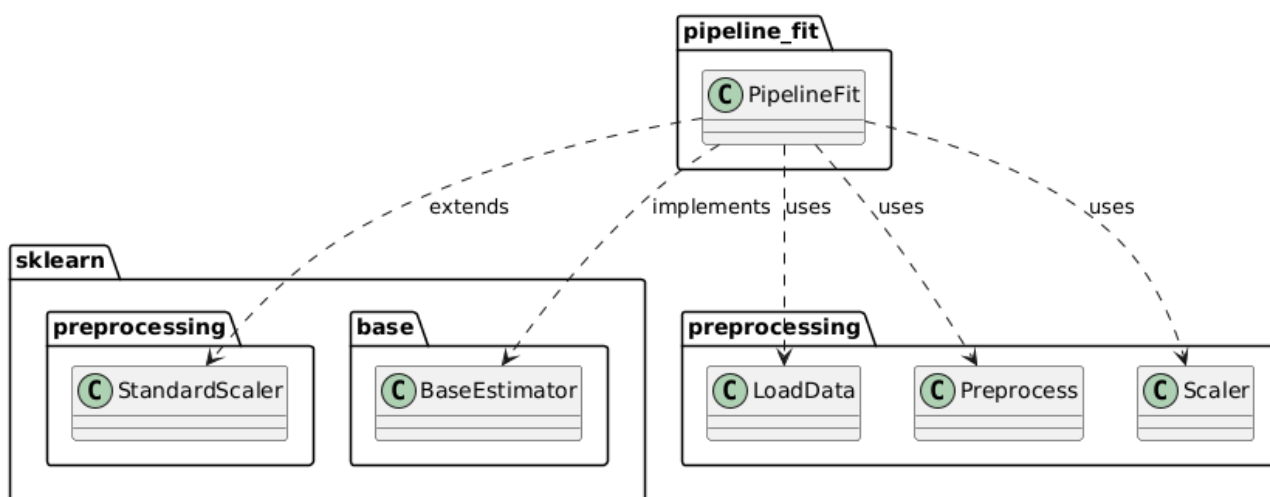


Рисунок 13 — Диаграмма классов модуля конвейера

Модуль конфигурации (файл `config.py`) отвечает за централизованное управление путями к хранилищам данных, инициализацию подключения к платформе управления экспериментами и настройку системы логирования. Вынесение всех зависимых от среды параметров в отдельный модуль обеспечивает принцип независимости кода от конфигурации (Configuration-as-Code) и упрощает развертывание системы на различных вычислительных узлах без необходимости изменения исходного кода.

Класс `Preprocess` отвечает за предобработку данных перед их подачей в нейросетевую архитектуру. Вариационные автокодировщики чувствительны к аномалиям во входных распределениях. Для решения этой задачи реализован класс `ScalerManager`, предоставляющий методы для обучения и

применения объекта `scaller` ко входным данным. Таким образом достигается стандартизация входящей информации.

Метод `delete_nan()` позволяет удалить строки во входном наборе признаков. Метод `marking_norm_anom()` реализует функцию маркировки нормальных и аномальных режимов. Данный метод служит дополнительной функцией для постановки экспериментов в ходе текущего исследования. В методе `marking_norm_anom_by_quantile()` реализована логика маркировки выбросов и пропусков на основе процентелей. Пример объявления метода представлен ниже.

```
def marking_norm_anom_by_quantile(
    self,
    dataframe: pd.DataFrame,
    quantile: float = 0.95,
    time_col: str = 'time in cycles',
    unit_col: str = 'unit number'
) -> pd.DataFrame
```

Метод `split_norm_anom()` позволяет отделить промаркированные штатные режимы от аномальных. В задачах прогнозирования остаточного ресурса аномальным режимом считается финальная стадия деградации двигателя, непосредственно предшествующая отказу. Разметка последних n циклов как аномальных соответствует физической природе процесса износа.

Метод `split_by_engine_train_test_val()` реализует стратегию разделения телеметрических данных на обучающую, валидационную и тестовую выборки по идентификаторам двигателей, а не по случайному распределению отдельных циклов. Данный подход критически важен для задач прогнозирования остаточного ресурса, поскольку предотвращает утечку информации между выборками, модель не увидит данные двигателей, которые будут использоваться для валидации и тестирования. Алгоритм данного метода представлен на рисунке 14.

Метод `create_sequences()` реализует стандартную стратегию нарезки непрерывных временных рядов на перекрывающиеся окна фиксированной длины для последующей подачи в нейросетевую модель. В отличие от специализированного метода `create_anomaly_sequences()`, который привязывает окна к аномальным циклам, данный метод формирует окна

Алгоритм разделения данных по идентификаторам двигателей

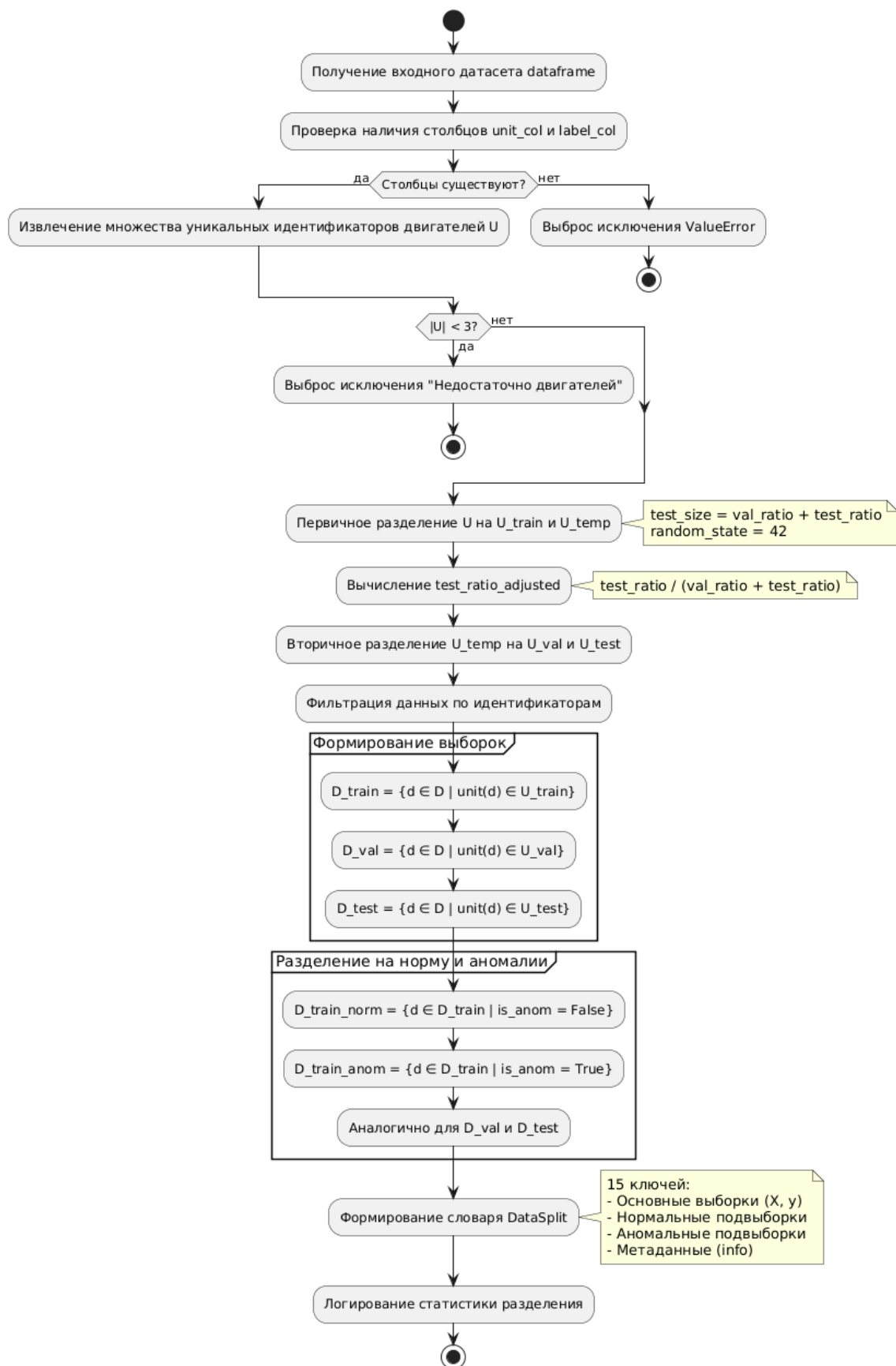


Рисунок 14 — Алгоритм разделения исходных данных метода `split_by_engine_train_test_val()`

равномерно по всей временной оси каждого двигателя, что обеспечивает покрытие всех фаз жизненного цикла — от штатной эксплуатации до предотказного состояния:

$$G = \text{groupby}(\text{dataframe}, \text{key} = \text{'unit number'}). \quad (50)$$

Фильтрация коротких двигателей:

$$\text{Если } N_g < \text{seq_length}, \text{ пропустить двигатель } g \quad (51)$$

Для каждого двигателя g с $N_g \geq \text{seq_length}$ необходимо:

1) вычислить количество окон:

$$B_g = \left\lfloor \frac{N_g - \text{seq_length}}{\text{stride}} \right\rfloor + 1; \quad (52)$$

2) определить длину окна для каждого начального индекса $j \in \{0, \text{stride}, 2 \cdot \text{stride}, \dots, N_g - \text{seq_length}\}$:

$$W_j = X_g[j : j + \text{seq_length}, :] \in \mathbb{R}^{\text{seq_length} \times D}; \quad (53)$$

3) добавить окно W_j в результирующий список $\mathcal{W}$.

Алгоритм метода создания окон из исходных данных представлена на рисунке 15.

Метод `apply_stress_to_data()` реализует комплексный механизм искусственного искажения телеметрических данных для оценки устойчивости генеративных моделей к аппаратным дефектам и зашумлению измерительной среды. Метод реализует два фундаментальных типа искажений, характерных для реальных промышленных систем мониторинга:

- аддитивный Гауссов шум, моделирующий вибрацию датчиков, электромагнитные наводки и погрешности измерений;
- случайные пропуски данных — моделируется потеря пакетов телеметрии, временный отказ датчиков, обрывы линий связи.

Подобная архитектура конвейера обеспечивает высокую надежность системы, гибкость настройки экспериментов и соответствие требованиям к воспроизводимости научных результатов.

Алгоритм формирования скользящих окон (create_sequences)

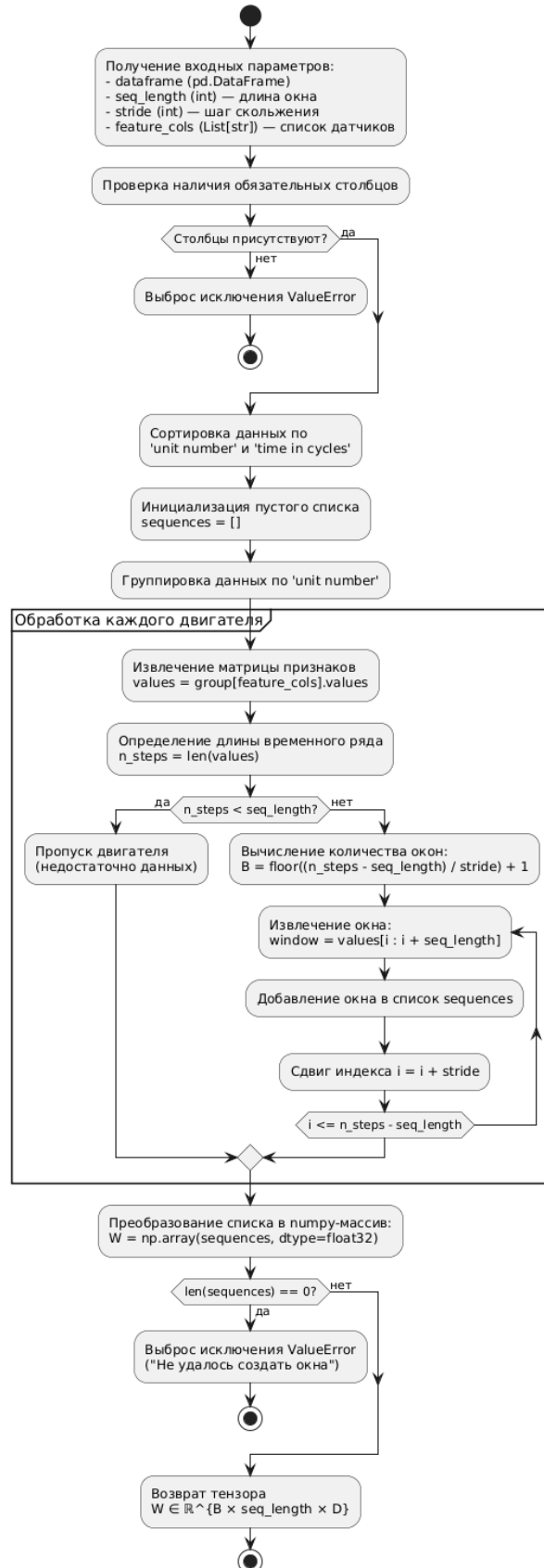


Рисунок 15 — Алгоритм искусственного зашумления и добавления пропусков к данным

Пусть $x_{i,d}$ — значение d -го датчика в i -м наблюдении исходной матрицы X_{clean} . Преобразование выполняется следующим образом.

Если параметр `noise_type` $\in \{\text{'noise'}, \text{'both'}\}$ и `noise_level` > 0 , то для каждого датчика $d \in \mathcal{S}$ (где $\mathcal{S}$ — множество индексов сенсорных колонок):

Диаграмма последовательности представлена на рисунке 16.

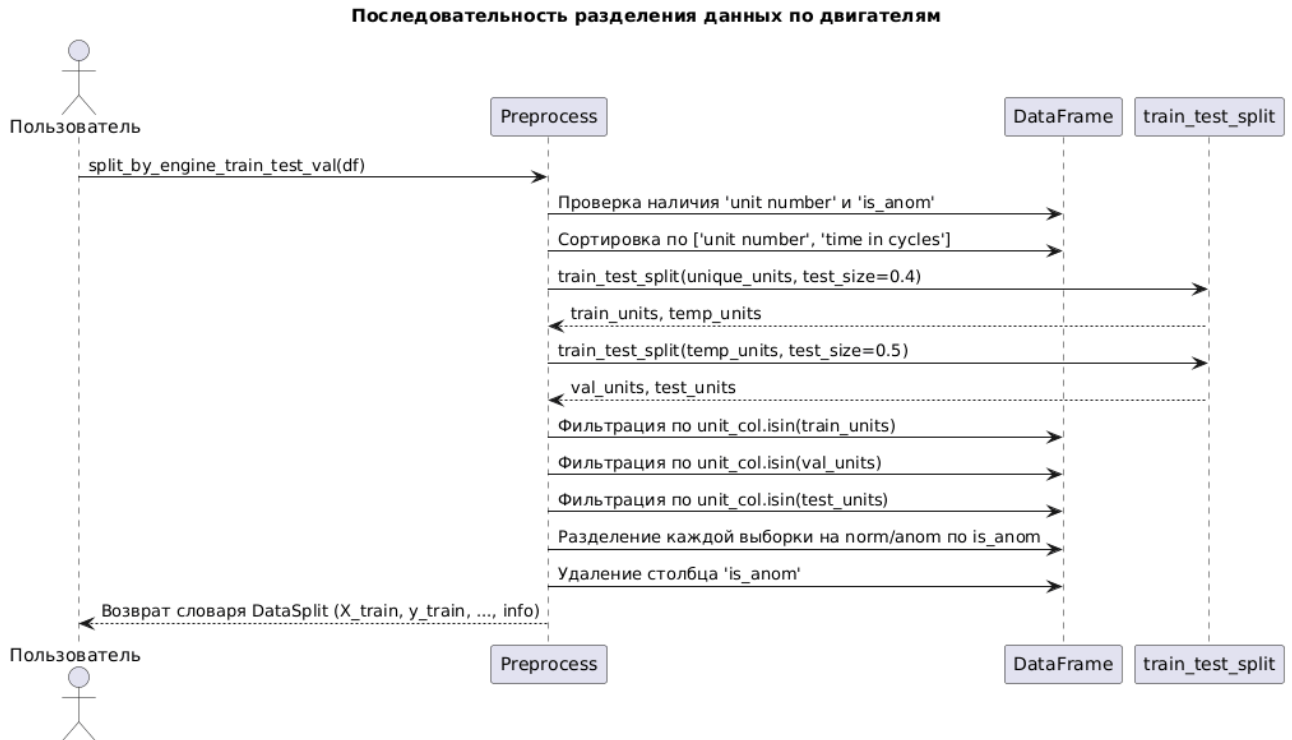


Рисунок 16 — Диаграмма последовательности разделения данных по двигателям

4.5 Модуль нейросетевых моделей

Модуль `models` является вычислительным ядром системы и содержит реализации генеративных нейросетевых архитектур на базе вариационного автокодировщика. Модуль обеспечивает полный жизненный цикл модели, от обучения на данных штатного режима до генерации вероятностных прогнозов и оценки неопределенности. Архитектура модуля построена на принципе единого интерфейса для всех реализованных моделей. Каждая архитектура наследует базовый класс `nn.Module` из PyTorch и реализует однообразный набор методов. На рисунке 17 изображена диаграмма классов `models`.

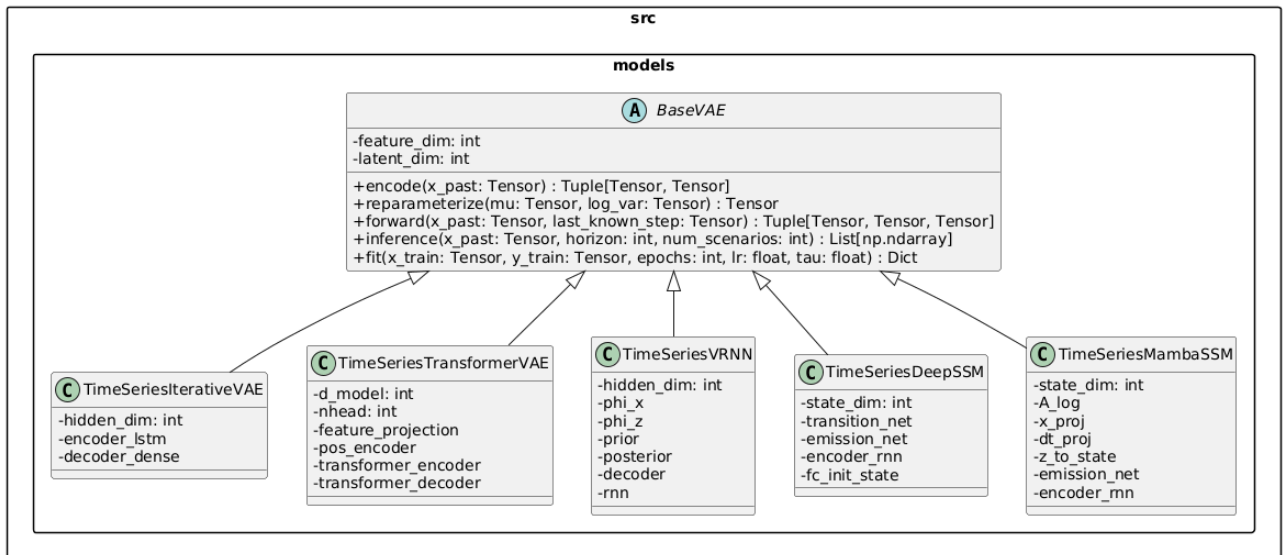


Рисунок 17 — Диаграмма классов на пакета models

Метод `encode()` является первым этапом работы архитектуры вариационного автокодировщика. Указанный метод — сжимает многомерную предысторию работы оборудования — окно из нескольких последовательных циклов с показаниями десятков датчиков в компактное стохастическое представление — параметры гауссовского распределения в латентном пространстве. На вход метода подаётся трёхмерный набор данных.

Метод пропускает входную последовательность через блок кодирования (это может быть LSTM, трансформер или их комбинация). Этот блок последовательно или параллельно обрабатывает все циклы окна и агрегирует информацию о состоянии оборудования в единое скрытое представление.

Результатом этого шага является финальное скрытое состояние (вектор фиксированной размерности, например 64 или 32 элемента), которое содержит сжатую информацию о всей предыстории: трендах деградации, колебаниях датчиков, общих паттернах поведения двигателя.

Финальное скрытое состояние пропускается через два независимых полносвязных слоя. Первый слой вычисляет вектор математического ожидания, являющийся центром латентного распределения, а второй — вычисляет вектор логарифма дисперсии, которая выполняет роль меры неопределённости в оценке состояния оборудования.

Оба вектора имеют размерность `latent_dim` (обычно 4 элемента), что на порядок меньше размерности входных данных. Метод возвращает кортеж из двух тензоров: `mu` и `log_var`, каждый размером `(batch_size, latent_dim)`.

Пример реализации метода `encode()` для LSTM-VAE архитектуры представлен ниже:

```
def encode(self, x_past):
    _, (h_n, _) = self.encoder_lstm(x_past)
    h_last = h_n[-1]
    mu = self.fc_mu(h_last)
    log_var = self.fc_log_var(h_last)
    return mu, log_var
```

Метод `reparameterize()` решает задачу генерации конкретного латентного вектора z из распределения, параметры которого являются математическое ожидание и дисперсия и были вычислены кодировщиком (метода `encode()`), при этом сохраняя возможность обратного распространения ошибки через операцию сэмплирования.

На вход метод принимает нижеуказанные параметры.

- математическое ожидание — вектор размерности `latent_dim`, определяющий центр гауссовского распределения в латентном пространстве;
- логарифм дисперсии — вектор той же размерности, определяющий «разброс» распределения вокруг центра.

Оба тензора имеют форму `(batch_size, latent_dim)`, где `batch_size` — количество независимых примеров, обрабатываемых параллельно.

Метод обеспечивает баланс между стохастичностью генерации через случайный шум и обучаемостью модели через детерминированные параметры.

Алгоритм данного метода представлен на рисунке 18. Метод `forward()` является центральным вычислительным узлом каждой из реализованных архитектур на базе вариационного автокодировщика. Данный метод решает задачу выполнения полного сквозного прохода данных через сеть, для предсказания одного следующего шага на этапе обучения.

Данный метод объединяет ранее описанные компоненты: кодировщик, механизм репараметризации и декодировщик, формируя итоговый прогноз, который затем сравнивается с истинными значениями набора данных для вычисления функции потерь.

Метод принимает нижеописанные ключевые аргументы.

- Предыстория — n -мерный тензор формы `(batch_size, seq_length,`

Алгоритм метода reparameterize

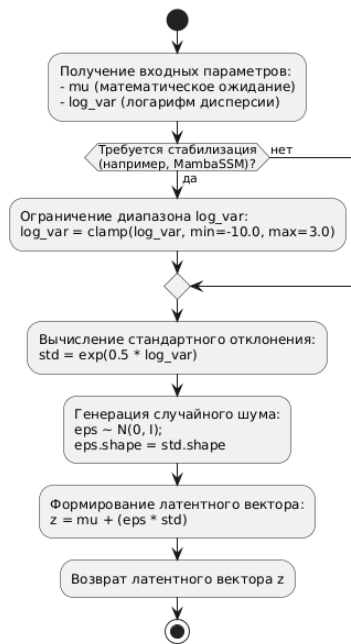


Рисунок 18 — Алгоритм работы метода reparameterize()

feature_dim), который является скользящим окном из последних n циклов работы двигателя, содержащее нормализованные показания датчиков;

- Точка опоры — двумерный тензор формы (batch_size, feature_dim), представляющий собой вектор показаний датчиков для самого последнего известного цикла из предыстории, служащий физической точкой отсчёта, от которой модель будет откладывать предсказанное приращение.

Метод возвращает кортеж из трёх элементов, необходимых для вычисления функции потерь.

- y_pred — тензор предсказанных значений датчиков для следующего шага, использующийся для вычисления ошибки реконструкции путём сравнения с истинными значениями;

- mu — вектор математического ожидания, использующийся для вычисления регуляризирующего члена KL-дивергенции;

- log_var — вектор логарифма дисперсии, который используется для вычисления KL-дивергенции, обеспечивая структурированность латентного пространства.

Метод inference() решает задачу генерации нескольких правдоподобных траекторий будущего состояния оборудования на заданном горизонте прогнозирования, начиная с известной предыстории.

В отличие от метода forward(), который предсказывает только один

следующий шаг для обучения, метод `inference()` выполняет многошаговую генерацию в замкнутом контуре, где каждый предсказанный шаг возвращается обратно в модель для прогнозирования следующего.

Ниже приведены три аргумента, передаваемые в метод `forward()`.

1) `x_past` (первые n циклов работы) — трёхмерный тензор формы `(batch_size, past_len, feature_dim)`, представляющий собой скользящее окно из последних известных циклов работы двигателя, содержащее нормализованные показания датчиков;

2) `horizon` (горизонт прогнозирования) — целое число, определяющее общую длину генерируемой последовательности (включая предысторию), если `past_len = 5` и `horizon = 10`, модель сгенерирует 5 будущих шагов;

3) `num_scenarios` (количество сценариев) — целое число, определяющее размер ансамбля генерируемых траекторий, где каждый сценарий представляет собой независимую реализацию будущего, что позволяет оценить неопределённость прогноза.

На каждом шаге метод `forward()` сэмплирует новый латентный вектор z из распределения. Это означает, что даже при одинаковой предыстории разные сценарии будут отличаться друг от друга, отражая стохастическую природу процесса деградации.

Каждый сгенерированный шаг возвращается обратно в модель для прогнозирования следующего. Это позволяет моделировать длинные траектории, но также приводит к накоплению ошибок на больших горизонтах.

Весь объем сценариев генерируются независимо друг от друга. Это обеспечивает статистическую репрезентативность ансамбля для оценки неопределённости.

Все сценарии начинаются с единой входной предыстории. Это гарантирует, что различия между сценариями обусловлены только стохастичностью будущего, а не различиями в начальных условиях. Диаграмма лгоритма работы метода `inference` представлена на рисунке 19.

Метод возвращает список `numpy`-массивов, где каждый элемент представляет собой один сгенерированный сценарий:

Метод имеет некоторые ограничения:

- накопление ошибок — поскольку каждый сгенерированный шаг

Алгоритм метода inference (Многошаговая авторегрессивная генерация)

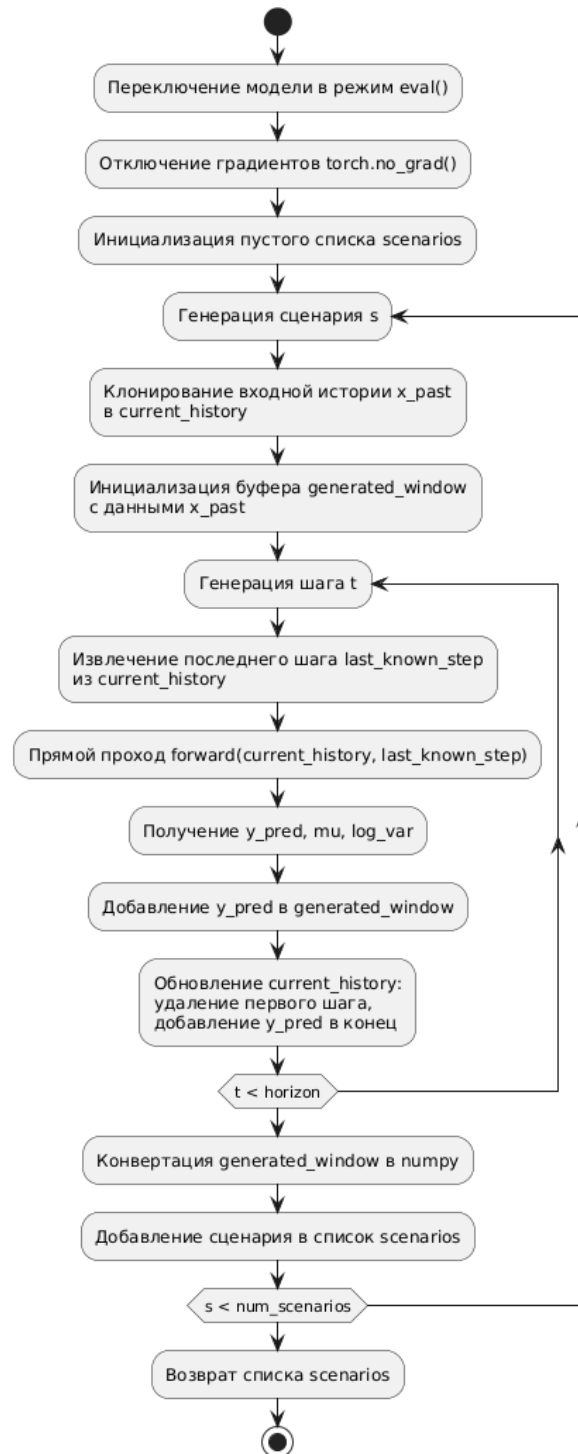


Рисунок 19 — Алгоритм работы метода reparameterize()

возвращается обратно в модель, ошибки накапливаются экспоненциально и на больших горизонтах качество прогноза может значительно деградировать;

- вычислительная сложность — при возрастании числа генерации большого количества сценариев на горизонте требуется кратное количество прямых проходов через модель, что может быть медленно для больших ансамблей.

Метод `inference()` обеспечивает практическое применение модели. Он преобразует стохастическую генеративную модель в инструмент для прогнозирования будущих траекторий с оценкой неопределённости. Ключевой особенностью метода является авторегрессивная генерация ансамбля независимых сценариев, что позволяет оценивать как наиболее вероятное будущее, так и спектр возможных отклонений от него.

Метод `fit()` обеспечивает обучение нейросетевой генеративной модели для одновременного точного восстанавливания наблюдаемых телеметрических сигналов и поддержания структурированного латентного пространства, пригодного для генерации правдоподобных сценариев. Алгоритм работы метода представлен на рисунке 20.

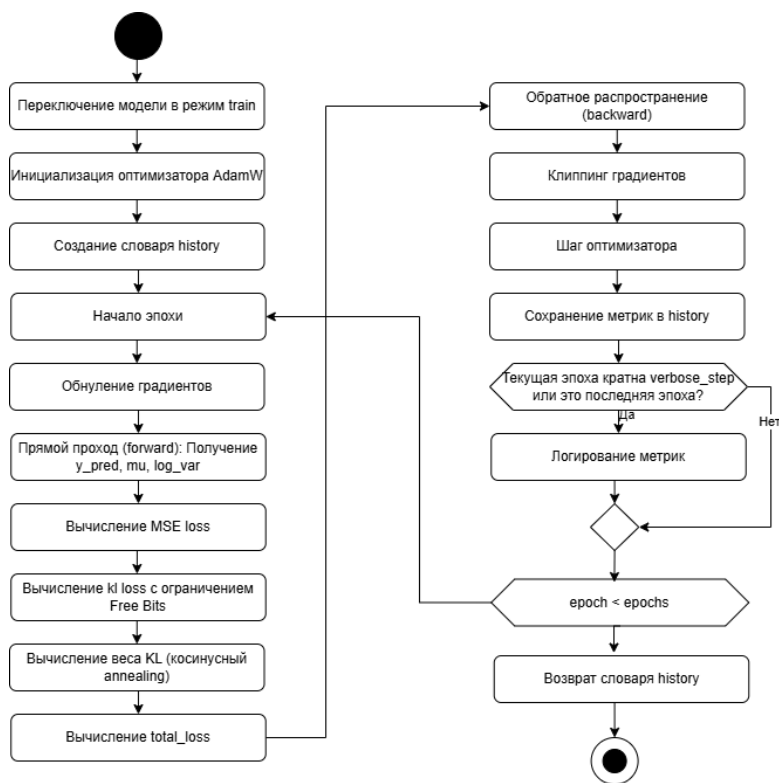


Рисунок 20 — Алгоритм работы метода `fit()`

Метод `fit()` реализует итеративный процесс оптимизации, в котором

на каждой эпохе модель делает предсказание, вычисляет ошибку, распространяет градиенты обратно по всем слоям и корректирует веса в направлении уменьшения ошибки.

Метод `fit()` позволяет проводить нижеописанные операции.

- Переключение в режим обучения — модель переводится в режим `train()`, что активирует механизмы, специфичные для обучения (например, вычисление градиентов).

- Инициализация оптимизатора — создается оптимизатор, автоматически адаптирующий скорость обучения для каждого параметра модели, что ускоряет сходимость.

- Создание контейнера истории — инициализируется словарь `history` с четырьмя пустыми списками для накопления метрик `total_loss`, `mse_loss`, `kl_loss`, `kl_weight` по эпохам.

Далее производится прямой проход. Модель реализует восстановления предыстории и генерацию последующих циклов. После чего происходит расчет ошибок реконструкции, генерации и поэлементной дивергенции Кульбака-Лейблера. Далее метод `fit()` проводит обратный проход, рассчитывая градиенты полной функции потерь по всем обучаемым параметрам модели. Градиенты показывают направление и величину изменения каждого веса для уменьшения ошибки. После чего оптимизатор AdamW обновляет все обучаемые параметры модели в направлении, противоположном градиентам, с учётом адаптивной скорости обучения и L2-регуляризации.

Диаграмма последовательности метода `fit()` представлена на рисунке 21.

4.6 Модуль оценки метрик

Модуль оценки метрик реализует пакет `src.metrics` и предоставляет унифицированный интерфейс для количественной оценки качества работы генеративной модели. В отличие от стандартных задач регрессии, оценка вариационного автокодировщика требует комплексного подхода, учитывающего точность поточечного прогнозирования, структурное соответствие временных рядов, качество детектирования аномалий и вероятностную согласованность модели. Структура классов модуля

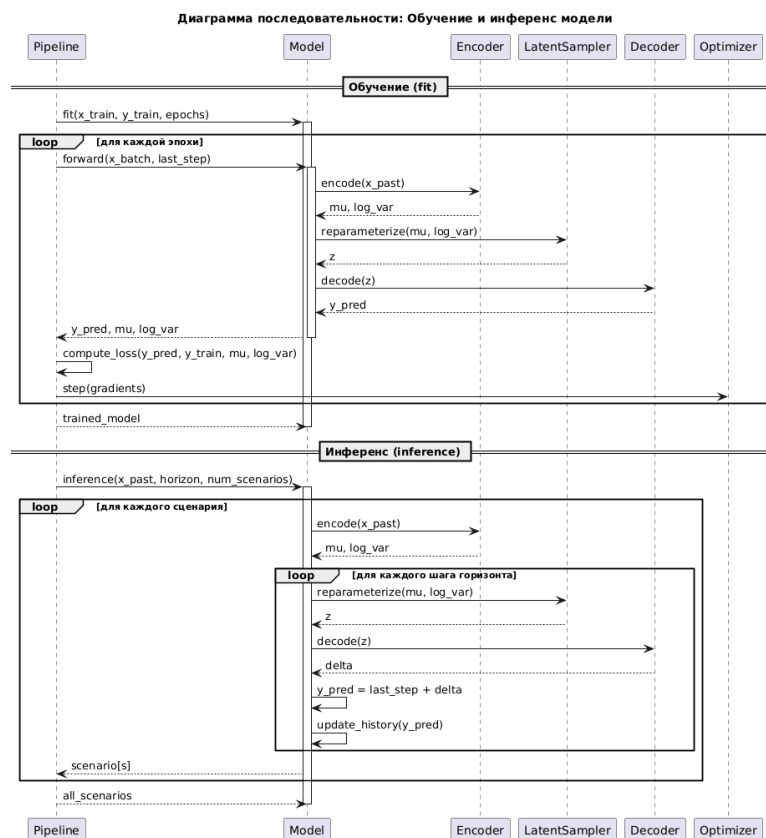


Рисунок 21 — Диаграмма последовательности пакета models

представлена на рисунке. Диаграмма классов данного модуля представлена на рисунке 23.

Класс `RegressionMetrics` реализует базовые метрики точности прогнозирования. Метод `compute_rmse()` вычисляет среднеквадратичную ошибку между сгенерированными траекториями и эталонными значениями, чувствительную к крупным отклонениям. Метод `compute_mae()` рассчитывает среднюю абсолютную ошибку, обладающую устойчивостью к редким выбросам. Метод `compute_r2()` определяет коэффициент детерминации, отражающий долю дисперсии телеметрических параметров, объясненную моделью.

Класс `TemporalMetrics` отвечает за оценку структурного соответствия временных рядов. Метод `compute_dtw()` реализует алгоритм динамической трансформации времени, который измеряет минимальное расстояние между двумя последовательностями с учетом нелинейных фазовых сдвигов. Это критически важно для сравнения траекторий деградации, которые могут развиваться с различной скоростью в разных двигателях. Метод `normalize_dtw()` выполняет нормализацию сырого значения метрики с

учетом длины последовательности, обеспечивая сопоставимость результатов для двигателей с разной продолжительностью жизненного цикла.

Класс `ClassificationMetrics` обеспечивает оценку качества детектирования аномалий на основе скалярного индикатора состояния S_t . Метод `compute_f1()` вычисляет гармоническое среднее точности и полноты классификации при заданном пороговом значении. Метод `compute_auc_roc()` рассчитывает площадь под ROC-кривой, характеризующую способность модели разделять штатные и аномальные режимы независимо от выбора конкретного порога. Метод `find_optimal_threshold()` автоматически определяет граничное значение индикатора S_t , максимизирующее метрику F1 на валидационной выборке.

Класс `NASAScore` реализует специализированную функцию потерь, принятую в датасете NASA C-MAPSS для оценки прогнозирования остаточного ресурса. Метод `compute()` применяет асимметричную штрафную функцию: запоздалые прогнозы отказа (когда модель предсказывает отказ позже фактического) штрафуются строже, чем преждевременные, поскольку первые несут большие риски для безопасности эксплуатации. Параметры `penalty_early()` и `penalty_late()` соответствуют коэффициентам 13 и 10 из оригинальной формулировке метрики.

Класс `ProbabilisticMetrics` предназначен для оценки качества генеративной модели. Метод `compute_elbo()` вычисляет значение вариационной нижней оценки на валидационной выборке, позволяя контролировать сходимость процесса обучения. Метод `compute_kl_divergence()` измеряет степень соответствия апостериорного распределения латентных переменных априорному стандартному нормальному распределению, что важно для обеспечения непрерывности и интерпретируемости скрытого пространства. Метод `compute_nll()` рассчитывает отрицательное логарифмическое правдоподобие, предоставляя альтернативную метрику качества вероятностной генерации.

Класс `StatisticalValidator` обеспечивает статистическую проверку значимости различий между архитектурами моделей. Метод `wilcoxon_test()` реализует непараметрический критерий Уилкоксона для связанных выборок, позволяющий сравнивать метрики двух моделей на одних и тех же тестовых двигателях без предположения о нормальном распределении ошибок.

Метод `compute_confidence_interval()` строит доверительный интервал для среднего значения метрики с заданным уровнем значимости. Метод `compute_effect_size()` вычисляет размер эффекта (коэффициент Коэна d) для оценки практической значимости наблюдаемых различий.

Диаграмма классов пакета метрик представлена на рисунке 22.

Таким образом, модуль оценки и расчета метрик обеспечивает анализ качества обучения и инференса модели, что дает возможность проводить статистическое сравнение исследуемых генеративных архитектур.

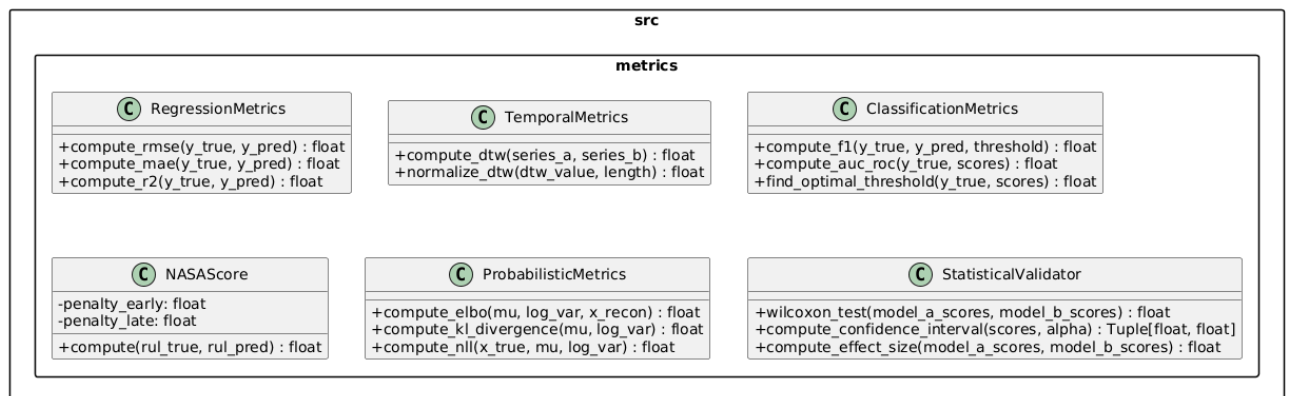


Рисунок 22 — Диаграмма классов модуля интеграции с MLFlow

4.7 Модуль интеграции с сервисом MLFlow и управления экспериментами

Модуль управления экспериментами реализует пакет `src.mlflowservices` и обеспечивает интеграцию системы с платформой MLflow. Данный модуль решает задачу автоматизации учета результатов исследования, версионирования обученных моделей и обеспечения воспроизводимости экспериментов. В условиях, когда процесс настройки гиперпараметров требует множества запусков, централизованное логирование позволяет отслеживать влияние изменений архитектуры на качество прогнозирования. Структура классов модуля представлена на рисунке 23.

Класс `MLflowClient` выступает в роли адаптера, инкапсулирующего логику подключения к серверу отслеживания. Метод `init_connection()` устанавливает соединение с удаленным хранилищем метаданных, а `set_experiment()` определяет логическую группу запусков, соответствующую конкретной серии экспериментов (например, подбор длины окна n или

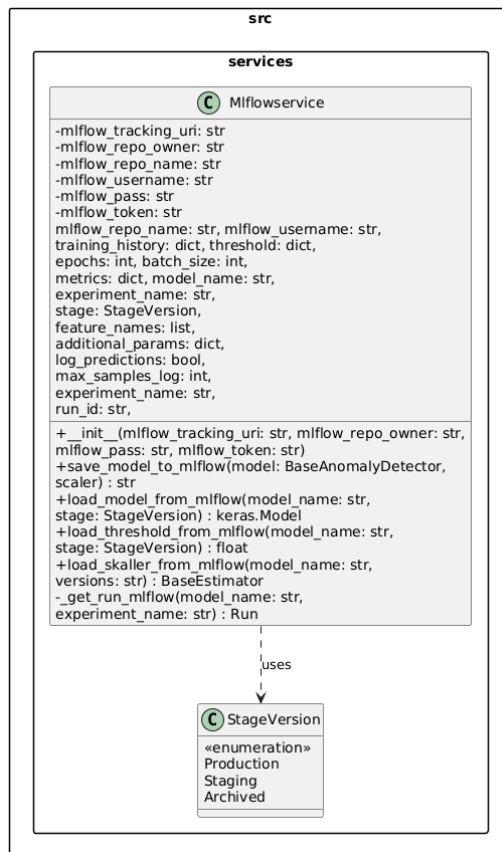


Рисунок 23 — Диаграмма классов модуля интеграции с MLFlow

коэффициента регуляризации λ). Это позволяет изолировать результаты различных этапов исследования друг от друга.

Метод `load_model_from_mlflow()` реализует загрузку обученной нейросетевой модели из централизованного реестра MLflow Model Registry. Метод обеспечивает возможность восстановления модели для проведения инференса или дообучения на новых данных, гарантируя использование проверенной конфигурации с соответствующей стадией жизненного цикла.

Метод выполняет три последовательных действия.

Установка URI трекинг-сервера — если в атрибутах экземпляра задан `mlflow_tracking_uri`, метод устанавливает его как активный URI для подключения к серверу MLflow:

```

if self.mlflow_tracking_uri:
    mlflow.set_tracking_uri(self.mlflow_tracking_uri);
  
```

Формирование URI модели — Метод формирует унифицированный идентификатор ресурса (URI) модели в формате MLflow. Данный формат позволяет адресовать модели как по стадии ("Production" "Staging"), так и по конкретному номеру версии ("3");

Загрузка модели. Метод вызывает `mlflow.keras.load_model(model_uri)` для десериализации модели из Artifact Store. В случае успешной загрузки метод возвращает объект `keras.Model`. При возникновении ошибок (неверный URI, отсутствие модели, проблемы сети) метод генерирует исключение `RuntimeError` с детальным описанием причины сбоя.

Диаграмма последовательности данного метода представлена на рисунке 24.

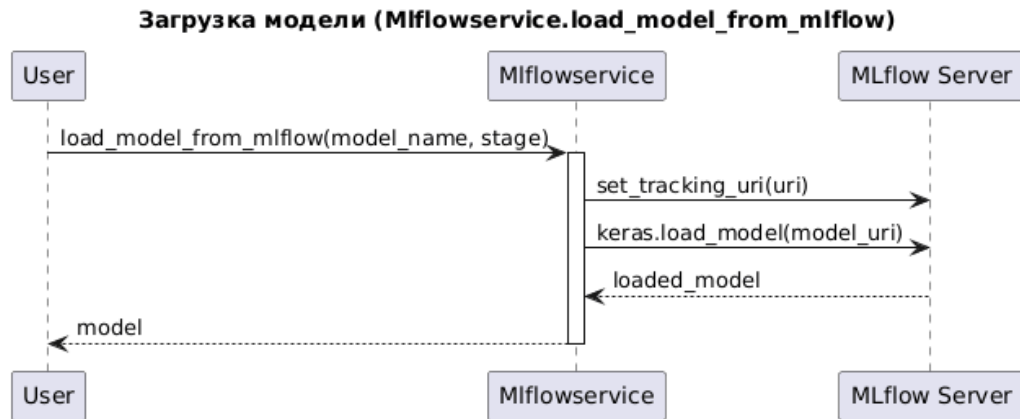


Рисунок 24 — Диаграмма последовательности работы метода `load_model_from_mlflow()`

Метод `save_model_to_mlflow()` реализует сохранение обученной модели, метрик обучения, калибровочного порога детектирования аномалий и объекта нормализации в централизованное хранилище MLflow. Метод обеспечивает полную трассировку эксперимента: все артефакты, гиперпараметры и метрики сохраняются в контексте одного запуска, что позволяет впоследствии восстановить точное состояние системы на момент обучения. диаграмма метода `save_model_to_mlflow()` представлена на рисунке 25.

Метод выполняет шесть последовательных этапов.

1) Установка эксперимента — метод вызывает `mlflow.set_experiment()` для создания или получения эксперимента по имени. Все последующие операции логирования будут привязаны к этому эксперименту.

2) Запуск метода `run()` — метод инициализирует новый запуск через контекстный менеджер: `with mlflow.start_run(run_name=f"{model_name}_run") as run:`. Внутри этого блока выполняются все операции логирования. По завершении блока `run` автоматически завершается.

- 3) Логирование параметров и метрик.
- 4) Логирование параметров и метрик.
- 5) Сохранение модели. Метод сериализует модель и сохраняет её в Artifact Store.

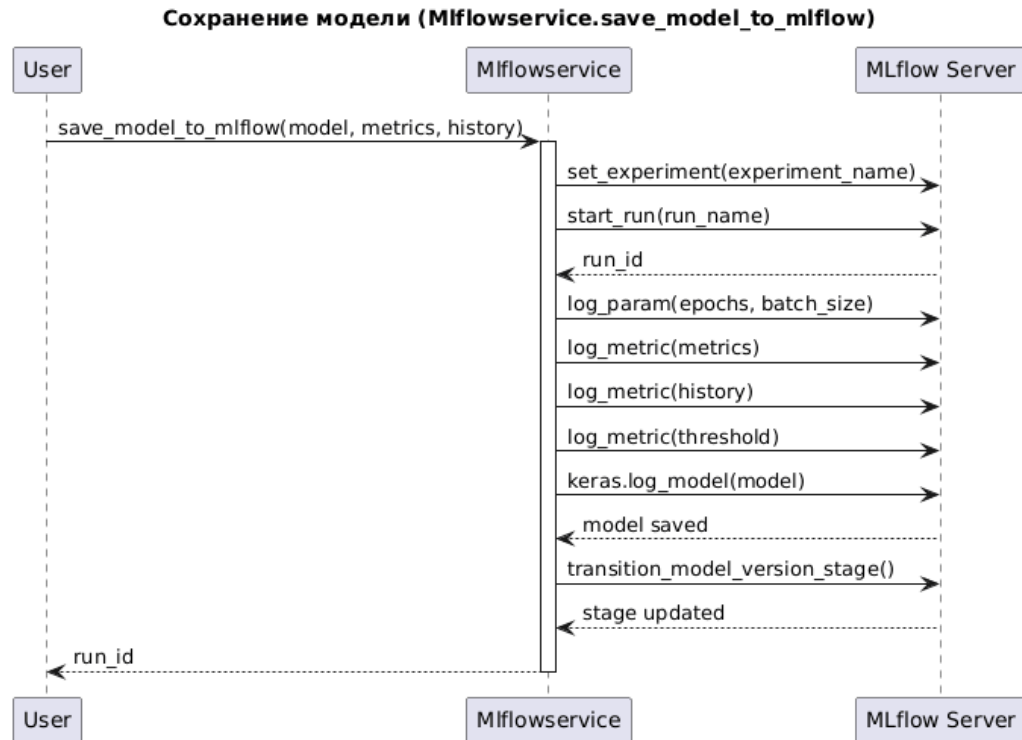


Рисунок 25 — Диаграмма последовательности работы метода `load_model_from_mlflow()`

В методе `load_skaller_from_mlflow()` реализуется загрузка объекта нормализации из реестра MLflow. Метод обеспечивает восстановление статистик нормализации (математическое ожидание и стандартное отклонение), вычисленных на обучающей выборке, что критически важно для корректной предобработки новых данных перед подачей в модель.

Интеграция данного модуля с конвейером обучения автоматизирует процесс документирования исследования. Все промежуточные результаты, включая калибровочные пороги индикатора S_t и статистики нормализации, сохраняются вместе с моделью, что исключает потерю контекста и упрощает развертывание системы в промышленную эксплуатацию.

Завершение описания программных модулей позволяет перейти к рассмотрению конкретных этапов функционирования системы и сценариев взаимодействия компонентов.

4.8 Описание применения модулей конвейера проведения экспериментов

Взаимодействие программных модулей в рамках единого конвейера обработки данных обеспечивается модулем `src.pipeline`. Последовательность вызовов компонентов обеспечивает строгую направленность потоков информации и предотвращает циклические зависимости. Логика выполнения конвейера представлена на диаграмме последовательности (рисунок 26).

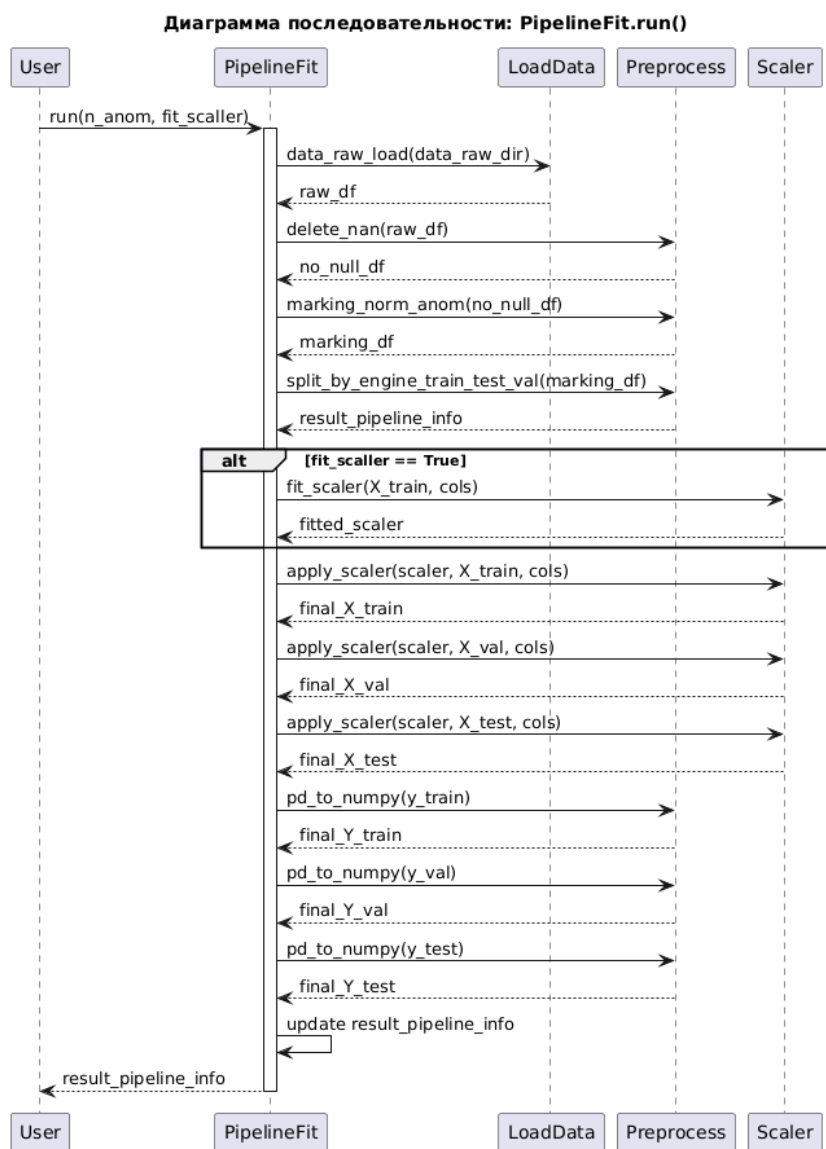


Рисунок 26 — Диаграмма последовательности взаимодействия компонентов системы

Процесс инициируется классом оркестратором, который загружает конфигурационный файл и проверяет соответствие параметров требованиям системы. Модуль чтения данных активируется в режиме генератора,

последовательно извлекая фрагменты телеметрии из внешних источников без блокировки основного потока выполнения. Полученные сырые массивы передаются в модуль предобработки, где выполняются операции интерполяции пропусков и фильтрации шумов. Далее применяется стандартизация на основе ранее вычисленных статистик, что гарантирует согласованность масштабов признаков.

Нормализованные данные направляются в функцию создания окон данных, которая агрегирует последовательные циклы в тензоры фиксированной размерности. Сформированные пакеты подаются в модуль генеративных моделей. В режиме обучения происходит итеративная оптимизация функции ELBO с вычислением градиентов и обновлением весов нейросети. Параллельно модуль метрик рассчитывает промежуточные показатели качества на валидационном подмножестве. По достижении критериев сходимости оптимизация завершается.

Финальные результаты выполнения конвейера фиксируются сервисом управления экспериментами. В хранилище артефактов сохраняются обученные веса модели, параметры нормализации и калибровочные пороги. Журнал событий пополняется записями о гиперпараметрах запуска и итоговых значениях метрик. Подобная синхронизация компонентов обеспечивает прозрачность процесса, возможность ретроспективного анализа и автоматизированное воспроизведение экспериментов при изменении конфигурации или входных данных.

4.9 Установка и развертывание системы

Разработанный программный комплекс реализует модульную архитектуру, ориентированную на гибкое развертывание в различных вычислительных средах — от локальных исследовательских стендов до промышленных серверов предиктивного обслуживания. Механизм доставки системы построен на принципах воспроизводимости окружения, изоляции зависимостей и автоматизации установки, что соответствует современным требованиям к промышленному программному обеспечению в области машинного обучения (MLOps).

Ключевым артефактом доставки является python-пакет

“modeling-work-system”, оформленный в соответствии со стандартом PEP 621 и скомпилированный через файл конфигурации `pyproject.toml`. Данный подход заменяет устаревшие механизмы на основе `setup.py` и обеспечивает декларативное описание метаданных проекта, зависимостей и процесса сборки.

Конфигурационный файл `pyproject.toml` содержит следующие ключевые параметры:

```
[project]
name = "modeling-work-system"
version = "0.4.0"
description = "Anomaly detection with autoencoders"
requires-python = ">=3.11".
```

Система имеет строго определённый набор внешних зависимостей.

```
dependencies = [
    "mlflow >=2.0"
    "numpy >=1.21"
    "pandas >=1.3"
    "scikit-learn >=1.0"
    "tensorflow >=2.10"
    "dagshub >=0.1.0"]
```

Файл `pyproject.toml` объявляет стандартные настройки сборки:

```
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"
```

Предлагаемый сценарий установки заключается в использовании программного ресурса PyPi и выполняется следующей командой:

```
pip install modeling-work-system==0.4.0
```

Так как система интегрируется с MLflow через клиентскую библиотеку dagshub, которая выступает адаптером к облачному хранилищу метаданных. Конфигурация подключения осуществляется через переменные окружения в файле формата `.env`. Пользователю необходимо указать актуальные данные целевого удаленного хранилища:

- MLFLOW_TRACKING_USERNAME;
- MLFLOW_TRACKING_PASSWORD;

- MLFLOW_TRACKING_TOKEN;

Подобный подход позволяет развернуть полный набор инструментов системы предиктивного обслуживания на сервере за несколько команд, обеспечивая идентичность среды разработки и промышленной эксплуатации.

4.10 Аппаратные и программные требования к работоспособности системы

Для эффективного функционирования системы моделирования сформулированы минимальные и рекомендуемые требования к вычислительным ресурсам.

Программные требования включают операционную систему семейства linux (Ubuntu 20.04 или новее), интерпретатор python версии 3.8 и выше, а также драйверы NVIDIA с поддержкой CUDA 11.0 или новее. Библиотечные зависимости (PyTorch, numpy, pandas, MLflow) устанавливаются автоматически в виртуальном окружении или контейнере.

Аппаратные требования зависят от режима работы. Для этапа обучения модели рекомендуется использование графического процессора NVIDIA с объемом видеопамяти не менее 2 ГБ (уровень GTX 1030). Это обусловлено необходимостью хранения градиентов и промежуточных активаций рекуррентных слоев при обработке длинных временных рядов. Для этапа инференса и прогнозирования достаточно центрального процессора с поддержкой инструкций AVX2 и 4 гигабайта оперативной памяти, так как генерация последовательностей не требует обратного распространения ошибки.

Требования к дисковой системе включают наличие быстрого накопителя (NVMe SSD) для временного хранения кэша данных и артефактов. Объем хранилища рассчитывается исходя из размера датасета и количества сохраняемых версий моделей, с рекомендуемым запасом не менее 128 ГБ для исследовательских задач.

4.11 Направления и возможности функционирования системы

Разработанный программный комплекс поддерживает три основных направления функционирования, соответствующих жизненному циклу предиктивного обслуживания технологического оборудования. Каждое направление реализовано в виде независимого программного конвейера, использующего общие модули обработки данных, моделирования и логирования. Подобная организация обеспечивает гибкость развертывания системы в различных условиях эксплуатации и позволяет адаптировать конфигурацию под требования конкретных промышленных объектов.

4.12 Выводы

В рамках четвертой главы выполнена программная реализация системы моделирования состояния технологического оборудования на базе нейросетевых генеративных моделей. Разработанный программный комплекс представляет собой законченное решение, охватывающее все этапы жизненного цикла предиктивного обслуживания: от приема сырых телеметрических сигналов до формирования вероятностных прогнозов и документирования результатов исследований. Проведенная работа позволила получить следующие основные результаты.

Спроектирована и реализована модульная архитектура программного комплекса, основанная на принципе разделения ответственности между функциональными слоями. Слой данных обеспечивает потоковое чтение и кэширование телеметрических рядов, слой предобработки выполняет очистку, нормализацию и агрегацию сигналов, слой моделирования содержит ядро системы — вариационный автокодировщик, слой метрик предоставляет унифицированный интерфейс для оценки качества, а слой сервисов интегрирует систему с платформой управления экспериментами. Подобная организация обеспечивает слабую связность компонентов, возможность их независимого тестирования и гибкое масштабирование при переходе к промышленной эксплуатации.

Создана открытая python-библиотека, структурированная в виде независимых пакетов с четкими интерфейсами взаимодействия. Пакет

`src.data` реализует механизм ленивой загрузки данных через генераторы, что позволяет обрабатывать массивы телеметрии объемом в гигабайты без полной выгрузки в оперативную память. Пакет `src.preprocessing` содержит классы для интерполяции пропусков, фильтрации шумов и Z-score стандартизации, причем статистики нормализации вычисляются исключительно на обучающей выборке для предотвращения утечки информации. Пакет `src.pipeline` координирует выполнение конвейера через класс оркестратор, обеспечивая валидацию входных параметров и логирование событий. Модульная структура библиотеки позволяет использовать отдельные компоненты автономно или в составе единого рабочего процесса.

Реализованы алгоритмы вариационного автокодировщика, строго соответствующие математической постановке задачи, сформулированной во второй главе. Класс `Encoder` аппроксимирует апостериорное распределение $q_\phi(z \mid X_{1:n})$ с использованием рекуррентных слоев для агрегации контекста временного ряда. Класс `LatentSampler` реализует трюк репараметризации, обеспечивая возможность обратного распространения ошибки через стохастический узел. Класс `Decoder` генерирует последовательность будущих циклов $\hat{X}_{n+1:n+k}$ на основе сэмплированного латентного вектора. Класс `ELBOLoss` вычисляет вариационную нижнюю оценку, объединяя член реконструкции и регуляризующую дивергенцию Кульбака-Лейблера. Программный код поддерживает гибкую настройку архитектуры скрытых слоев и гиперпараметров оптимизации.

Разработан комплекс метрик для всесторонней оценки качества моделирования. Модуль `RegressionMetrics` вычисляет поточечные ошибки RMSE, MAE и коэффициент детерминации. Модуль `TemporalMetrics` реализует метрику динамической трансформации времени (DTW) для учета фазовых сдвигов в траекториях деградации. Модуль `ClassificationMetrics` оценивает качество детектирования аномалий через метрики F1 и AUC-ROC. Модуль `NASAScore` применяет специализированную асимметричную функцию потерь для оценки прогнозирования остаточного ресурса. Модуль `ProbabilisticMetrics` контролирует сходимость обучения через значения ELBO и дивергенции Кульбака-Лейблера. Модуль `StatisticalValidator` обеспечивает статистическую проверку гипотез с использованием непараметрических

критериев. Подобный набор критериев позволяет объективно сравнивать разработанные решения с существующими аналогами.

Настроены автоматизированные конвейеры для трех основных сценариев функционирования системы. Конвейер обучения координирует итеративную оптимизацию функции потерь, валидацию на отложенном подмножестве и сохранение артефактов в реестр моделей. Конвейер инференса выполняет загрузку актуальной версии модели, предобработку новых данных, генерацию прогнозов и вычисление вероятности отказа. Конвейер валидации обеспечивает комплексную проверку работоспособности системы на тестовых выборках с формированием отчетов о качестве. Интеграция с платформой MLflow автоматизирует логирование гиперпараметров, метрик и версий моделей, гарантируя полную воспроизводимость экспериментов при повторных запусках.

Сформулированы требования к аппаратному и программному обеспечению для развертывания системы. Программные требования включают операционную систему Linux, интерпретатор python версии 3.11 и выше, драйверы CUDA и набор библиотек для научных вычислений. Аппаратные требования дифференцированы для режимов обучения (рекомендуется графический процессор с памятью от 8 ГБ) и инференса (достаточно центрального процессора).

Разработанный программный комплекс прошел внутреннюю проверку на корректность обработки граничных условий, согласованность размерностей тензоров и устойчивость к зашумлениям и выбросам во входных данных. Реализованные алгоритмы демонстрируют соответствие теоретическим ожиданиям: функция потерь ELBO монотонно возрастает в процессе обучения, латентное пространство сохраняет непрерывную структуру, а сгенерированные траектории воспроизводят характерные паттерны телеметрических сигналов. Созданная архитектура, модульная организация кода и автоматизированные конвейеры создают надежную техническую базу для проведения экспериментального исследования.

ЗАКЛЮЧЕНИЕ

В рамках выполненной работы по разработке, системы моделирования данных сложного технологического оборудования были достигнуты все поставленные цели и решены ключевые научно-инженерные задачи. На основе полученных экспериментальных и теоретических результатов сформулированы следующие основные выводы.

Разработана и успешно верифицирована программная топология модели вариационного автокодировщика с использованием фреймворка PyTorch без привлечения сторонних низкоуровневых скомпилированных ядер.

Обнаружено, что применение алгоритмов моделирования с использованием нейросетевых генеративных моделей на основе архитектуры вариационного автокодировщика позволяет снизить временную вычислительную сложность относительно длины обрабатываемых окон, что позволяет проводить обучение глубоких генеративных моделей в условиях жестких аппаратных ограничений и дефицита видеопамати графического процессора.

Проведен комплексный сравнительный анализ разработанных моделей с классами глубоких последовательных модулей, включая сети внимания, вариационные рекуррентные цепи, трансформеры с механизмом внимания. Установлено, что модель VRNN-VAE способна воспроизводить физическую природу износа авиационных двигателей, воссоздавая ступенчатый, скачкообразный характер накопления локальных повреждений, в то время как аналоги склонны к избыточному сглаживанию и затуханию трендов.

В ходе проведения многошагового инференса на полный цикл жизни двигателя в условиях высокочастотного зашумления и одновременного случайного пропуска пакетов данных модель VRNN-VAE продемонстрировала удовлетворительный уровень устойчивости. На больших горизонтах слепого прогнозирования модель генерировала устойчивые прогнозы будущих циклов, зафиксировав ошибку в пределах среднеквадратичного значения 0.6949, что на 3.2 процента превосходит показатели модели на базе долгой краткосрочной памяти. Разработанные методы центрирования приращений и определения точки опоры позволили

достичь положительного коэффициента детерминации, подтвердив перспективность применения разработанной архитектуры для точной генерации будущих циклов работы в условиях наличия зашумленных и пропусков в технологических средах.

Была разработана система моделирования данных. В данной системе были реализованы следующие программные пакеты:

- пакет загрузки и обработки данных, позволяющий читать данные в формате CSV и JSON, предоставляет классы для предобработки загруженных данных и применение класса нормализации и стандартизации данных;
- пакет, содержащий готовый конвейер обработки данных, позволяющий пользователю получить набор данных, готовый для обучения модели или проведения инференса;
- пакет-хранилище готовых нейросетевых моделей;
- пакет предоставляющий интеграцию с сервисом MLFlow, предоставляющий возможность сохранять проведенные пользователем эксперименты в удаленном хранилище, а также загружать предобученные модели из репозитория;

В результате проведенного исследования разработана и опробирована система моделирования состояния сложного технологического оборудования на основе вариационного автокодировщика. Система обеспечивает удовлетворительное прогнозирование остаточного ресурса, устойчивую работу в условиях зашумленного потока входных данных и возможность вероятностной оценки неопределённости прогноза. Разработанный программный комплекс и методология готовы к практическому внедрению в промышленных системах предиктивного обслуживания.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1) Kingma, D. P. Auto-Encoding Variational Bayes / D. P. Kingma, M. Welling // International Conference on Learning Representations (ICLR). – 2014. – 14 p. – URL: <https://arxiv.org/abs/1312.6114> (дата обращения: 17.04.2026).
- 2) Гурина, А. О. Обнаружение аномальных событий на хосте с использованием автокодировщика / А. О. Гурина, О. Ю. Гузев, В. Л. Елисеев // Системы контроля окружающей среды. – 2018. – № 3. – С. 12–19.
- 3) Chung, J. A Recurrent Latent Variable Model for Sequential Data / J. Chung, K. Kastner, L. Dinh, K. Goel, A. C. Courville, Y. Bengio // Advances in Neural Information Processing Systems (NeurIPS). – 2015. – Vol. 28. – P. 2980–2988. – URL: <https://proceedings.neurips.cc/paper/2015/hash/d7322ed717dedf1eb4e6e52a37ea7bcd-Abstract.html> (дата обращения: 15.01.2026).
- 4) Zheng, S. Variational Autoencoder for Remaining Useful Life Prediction / S. Zheng, K. Ristovski, A. Farahat, C. Gupta // 2019 IEEE International Conference on Prognostics and Health Management (ICPHM). – 2019. – P. 1–8. – DOI: 10.1109/ICPHM.2019.8819432.
- 5) Карпенко, А. П. Современные алгоритмы машинного обучения и анализа данных / А. П. Карпенко. – Москва : Изд-во МГТУ им. Н. Э. Баумана, 2020. – 288 с. – ISBN 978-5-7038-5345-2.
- 6) Guo, J. An Anomaly Detection Framework Based on Autoencoder and Nearest Neighbor / J. Guo, G. Liu, Y. Zuo, J. Wu // IEEE Access. – 2020. – Vol. 8. – P. 113588–113599. – DOI: 10.1109/ACCESS.2020.3003162.
- 7) Goodfellow, I. Deep Learning / I. Goodfellow, Y. Bengio, A. Courville. – Cambridge, MA : MIT Press, 2016. – 775 p. – ISBN 978-0-262-03561-3. – URL: <https://www.deeplearningbook.org> (дата обращения: 15.04.2026).
- 8) Sohn, K. Learning Structured Output Representation using Deep Conditional Generative Models / K. Sohn, H. Lee, X. Yan // Advances in Neural Information Processing Systems (NeurIPS). – 2015. – Vol. 28. – P. 3483–3491.
- 9) Borji, A. Pros and Cons of GAN Evaluation Measures / A. Borji // Computer Vision and Image Understanding. – 2022. – Vol. 219. – P. 103411. – DOI: 10.1016/j.cviu.2022.103411. – URL: <https://arxiv.org/abs/2107.02559> (дата обращения: 25.04.2026).

- 10) Li, X. Remaining useful life estimation in prognostics using deep convolution neural networks / X. Li, W. Zhang, Q. Ding // Reliability Engineering and System Safety. – 2018. – Vol. 172. – P. 1–11. – DOI: 10.1016/j.ress.2017.11.021.
- 11) Chauhan, S. Anomaly detection for multivariate time series through modeling temporal dependence of stochastic variables / S. Chauhan, L. M. Vig // Proceedings of the 21st ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. – 2015. – P. 93–102. – DOI: 10.1145/2783258.2788613.
- 12) Saxena, A. Data-Driven Prognostics Using a Kalman Filter Ensemble of Neural Network Models / A. Saxena, K. Goebel, D. Simon, N. Eklund // 2008 International Conference on Prognostics and Health Management. – 2008. – P. 1–10. – URL: https://ti.arc.nasa.gov/m/profile/adeq/data/PHM08_Challenge_Data_Description.pdf (дата обращения: 19.04.2026).
- 13) Hochreiter, S. Long Short-Term Memory / S. Hochreiter, J. Schmidhuber // Neural Computation. – 1997. – Vol. 9, № 8. – P. 1735–1780. – DOI: 10.1162/neco.1997.9.8.1735.
- 14) Хайкин, С. Нейронные сети: полный курс / С. Хайкин ; пер. с англ. – 2-е изд. – Москва : Вильямс, 2006. – 1104 с. – ISBN 978-5-8459-0960-5.